

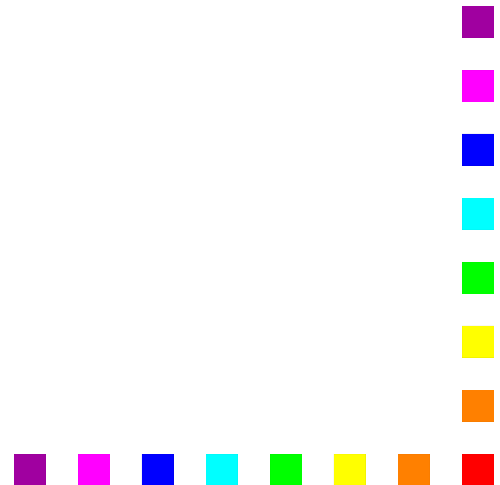
Introduction to Diagrammatic Stepwise Refinement

DIAGRAMMATIC STEPWISE REFINEMENT (DSR).

an approach to program design based on

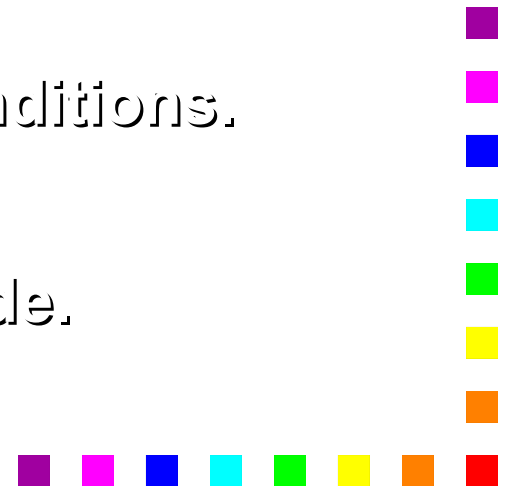
The Program Structure Diagrams used in the Jackson Method (JSP)

Wirth's Stepwise Refinement approach to program design.



Steps in the DSR approach

1. Define the problem completely.
2. Work out what needs to be done to solve the problem.
3. Using Stepwise Refinement, design a Program Structure Diagram (**PSD**) to represent your solution to the problem.
4. Write out the Executable Operations needed to produce the output from the input.
5. Assign the **Executable Operations** to the appropriate places in the PSD.
6. Insert the **Iteration** and **Selection** conditions.
7. Test your solution using test data.
8. Translate the populated PSD into code.

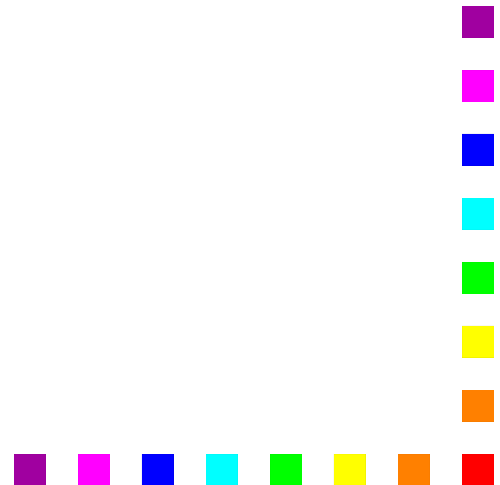


Stepwise Refinement

Is an approach to program design which involves breaking a task into sub-tasks.

Each sub-task is in turn broken into further sub-tasks.

This process continues until the sub-tasks are so simple that their implementation is straight forward.



Program Structure Diagrams

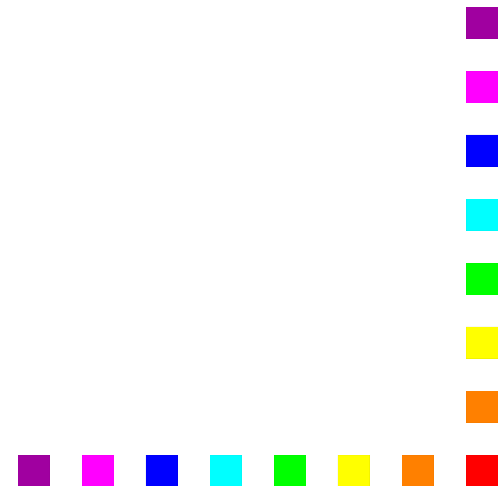
Program Structure Diagrams (PSDs) are used to represent the hierarchy of tasks and sub-tasks created by Stepwise Refinement.

They are drawn using the three classic constructs of Structured Programming.

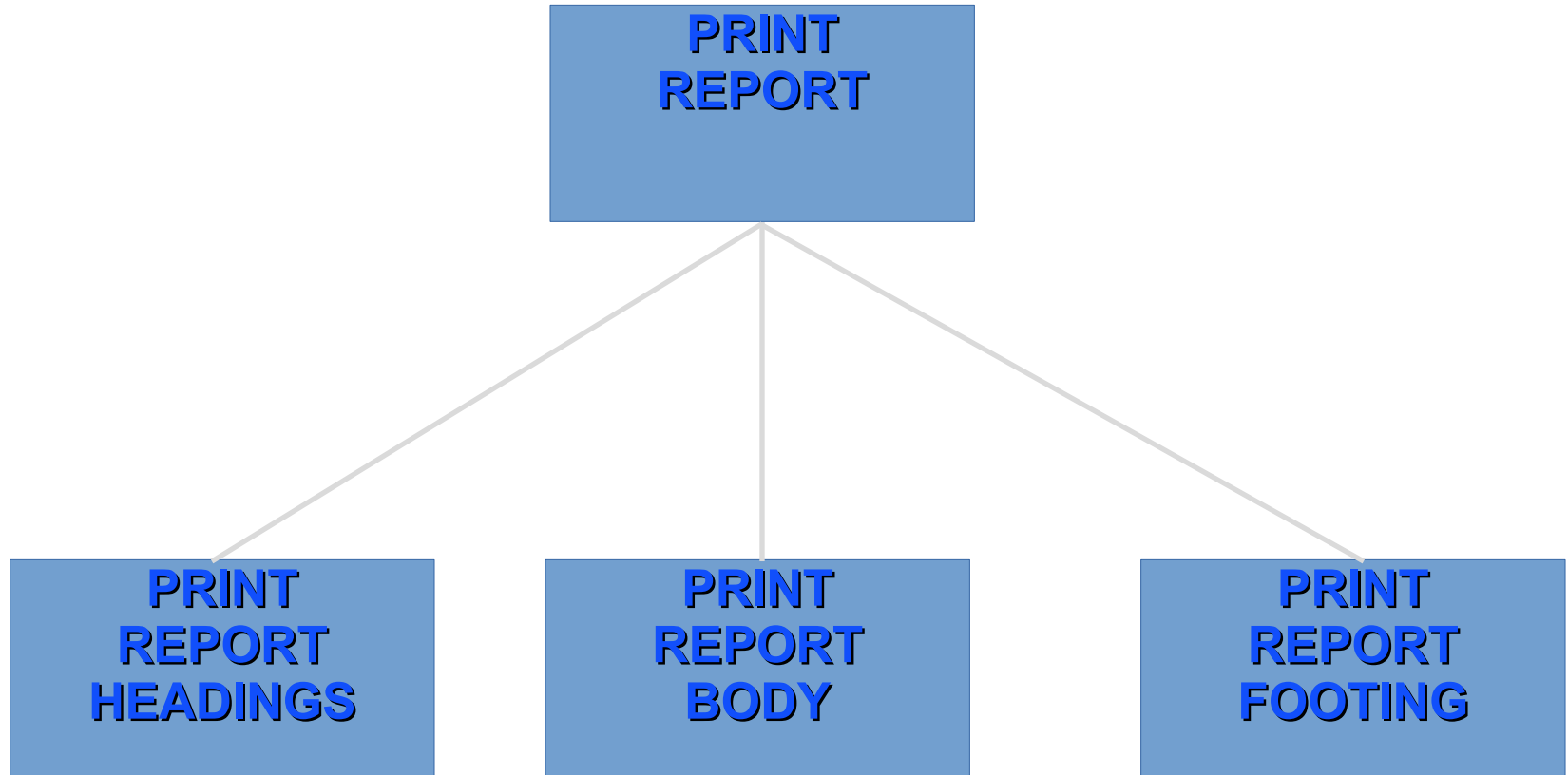
Sequence

Selection

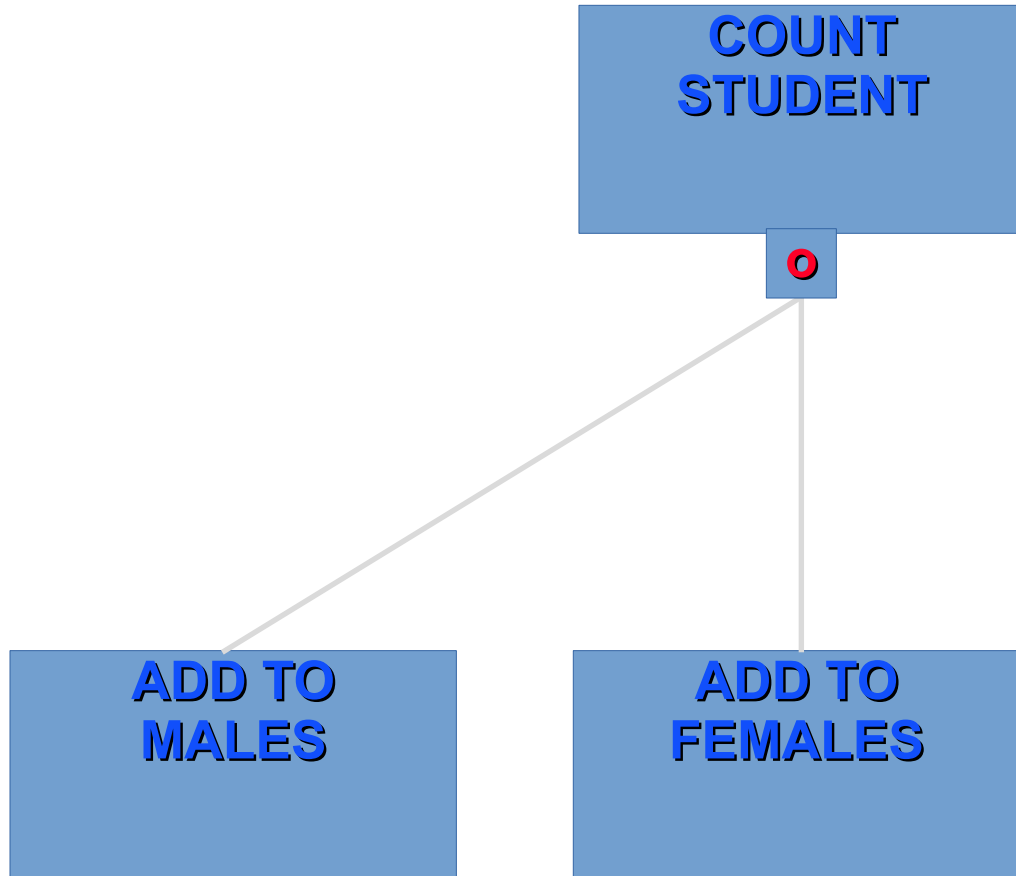
Iteration



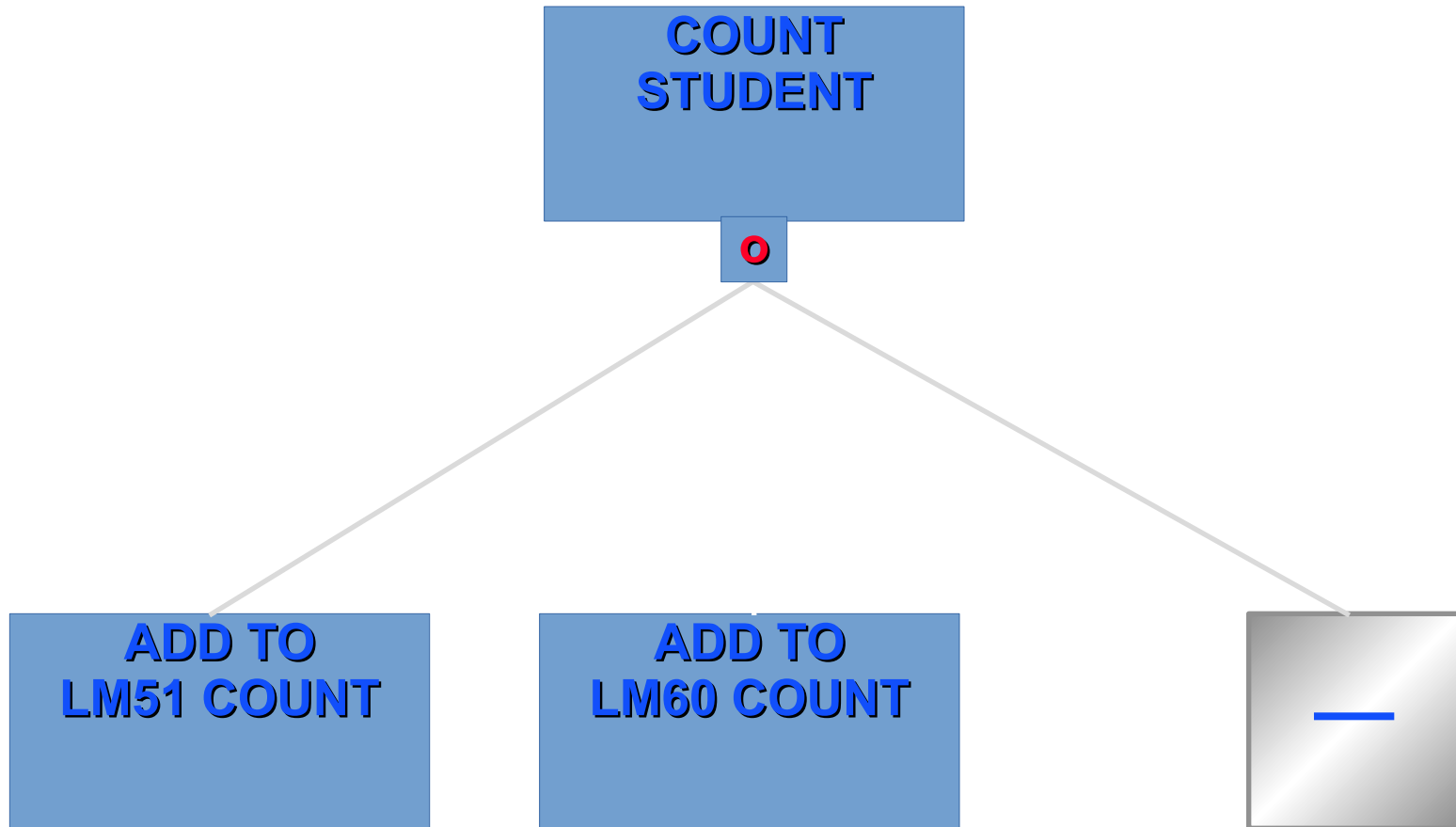
Sequence PSD



Selection PSD



Selection with Null part



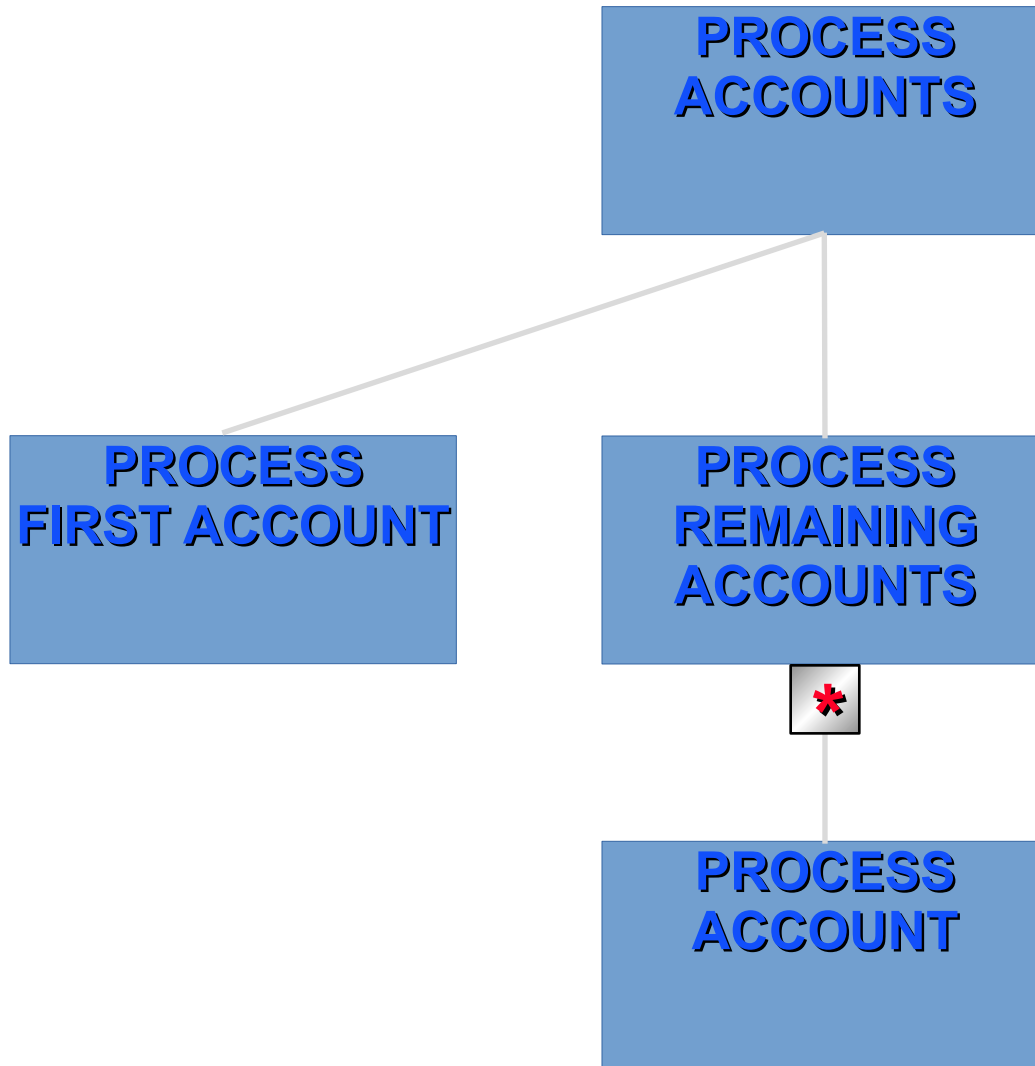
Iteration PSD

**PROCESS
ACCOUNTS**

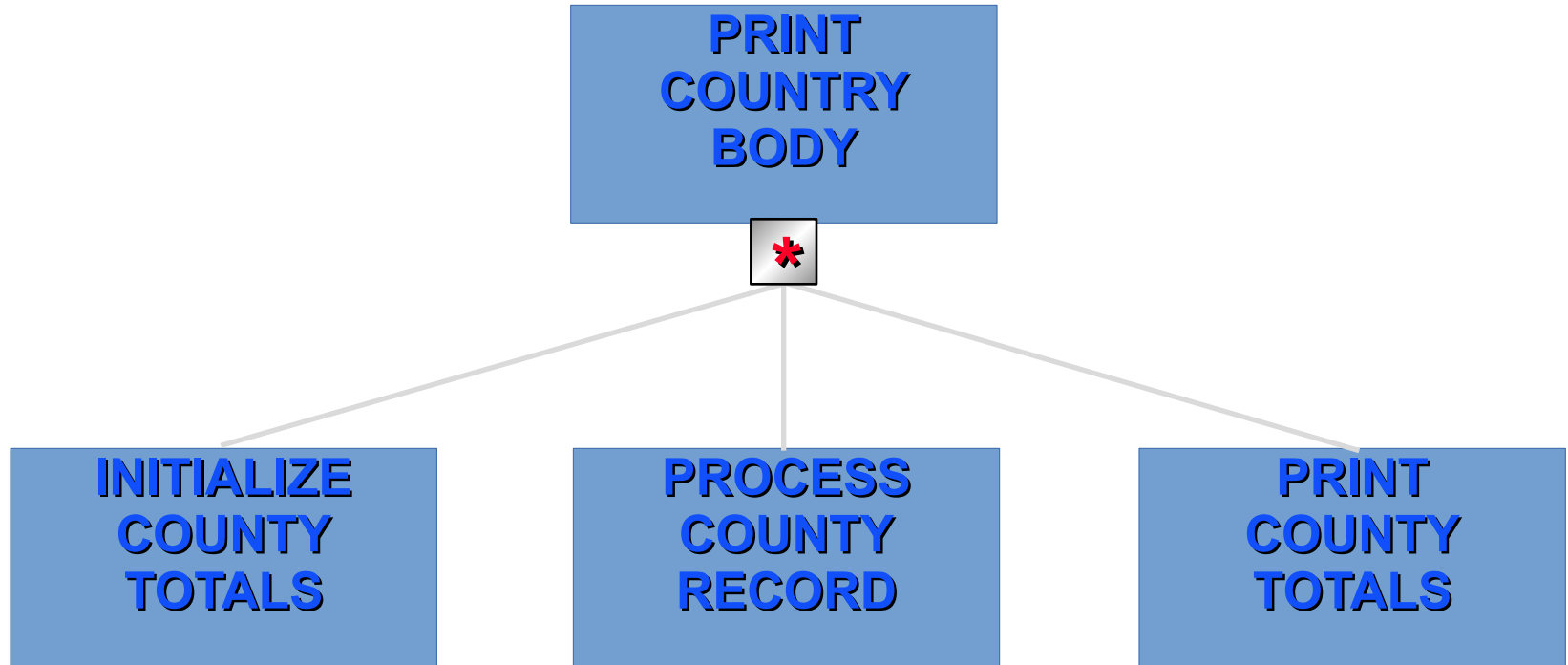


**PROCESS
ACCOUNT**

Iteration PSD - Explicit Repeat Loop



Iteration PSD

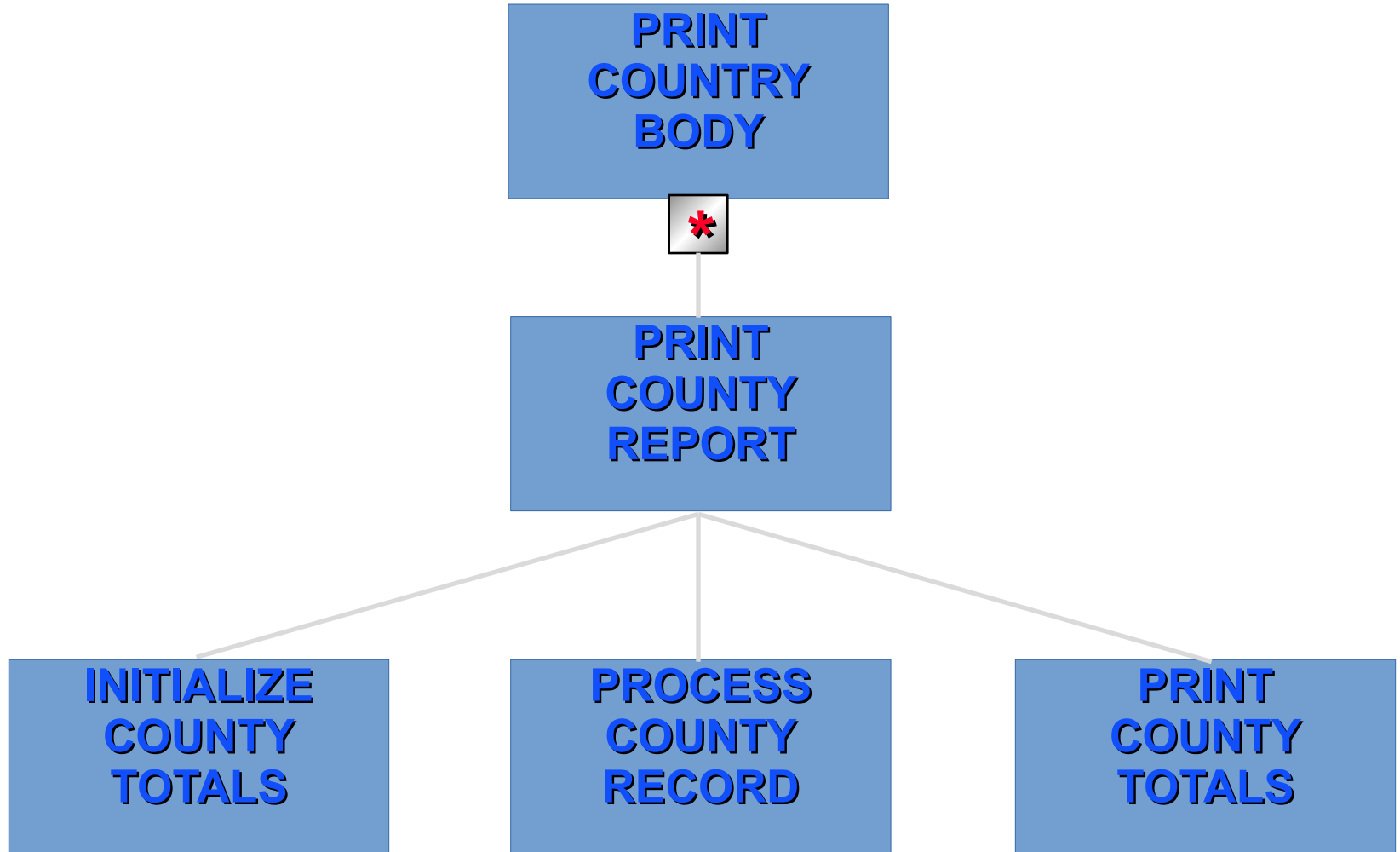


Print-Country-Body is an iteration of three components.

This is equivalent to ;

an iteration of one component which is in turn a sequence of three.

Iteration PSD



Payroll Proof Totals Program Specification

Write a program to produce proof totals for a weekly payroll. The payroll file consists of records with the following format:-

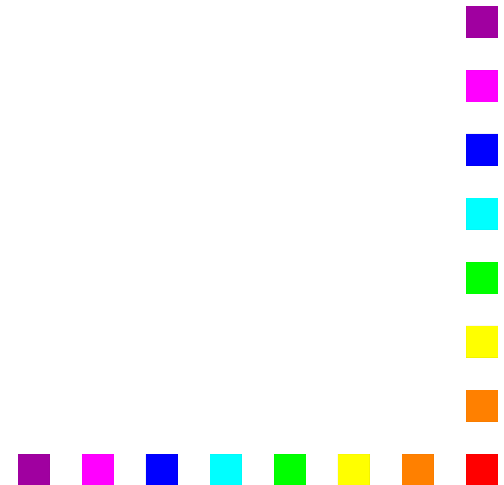
The print layout should be as follows:

PAYROLL PROOF TOTALS

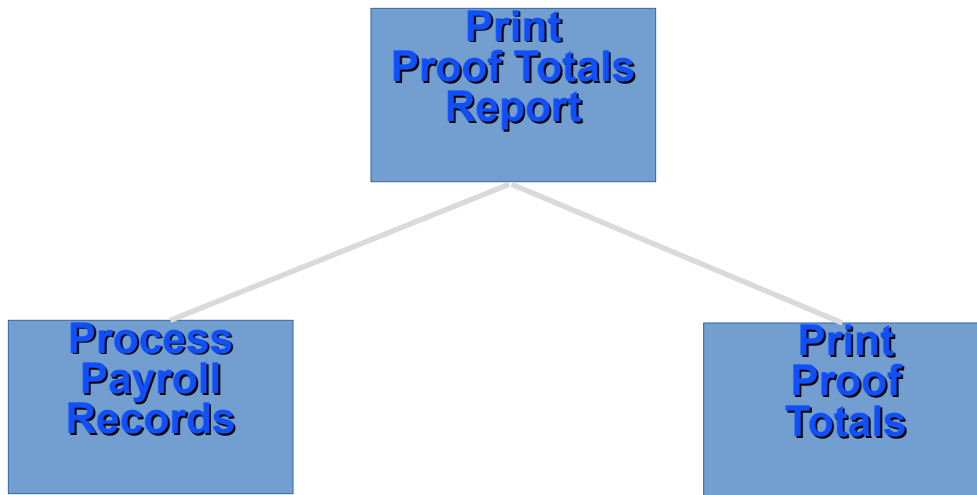
Regular Earnings :	25	\$2,111.23
Overtime Earnings :	2	\$21.50
Bonus Earnings :	6	\$123.45

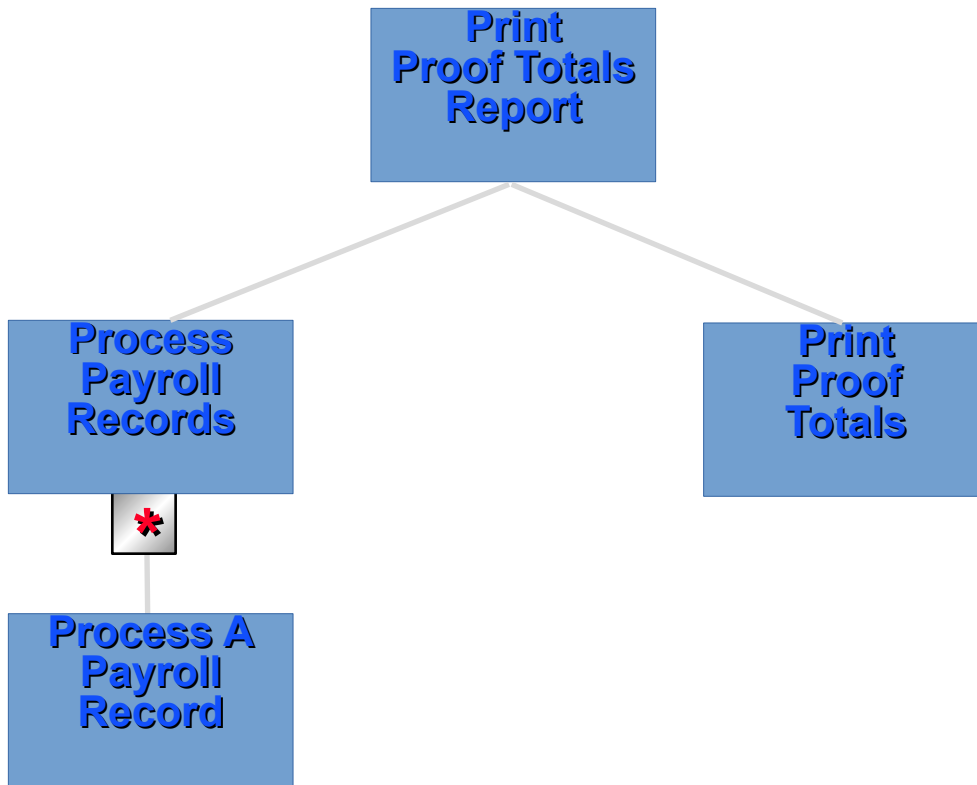
Applying DSR to the Payroll Proof Totals Program

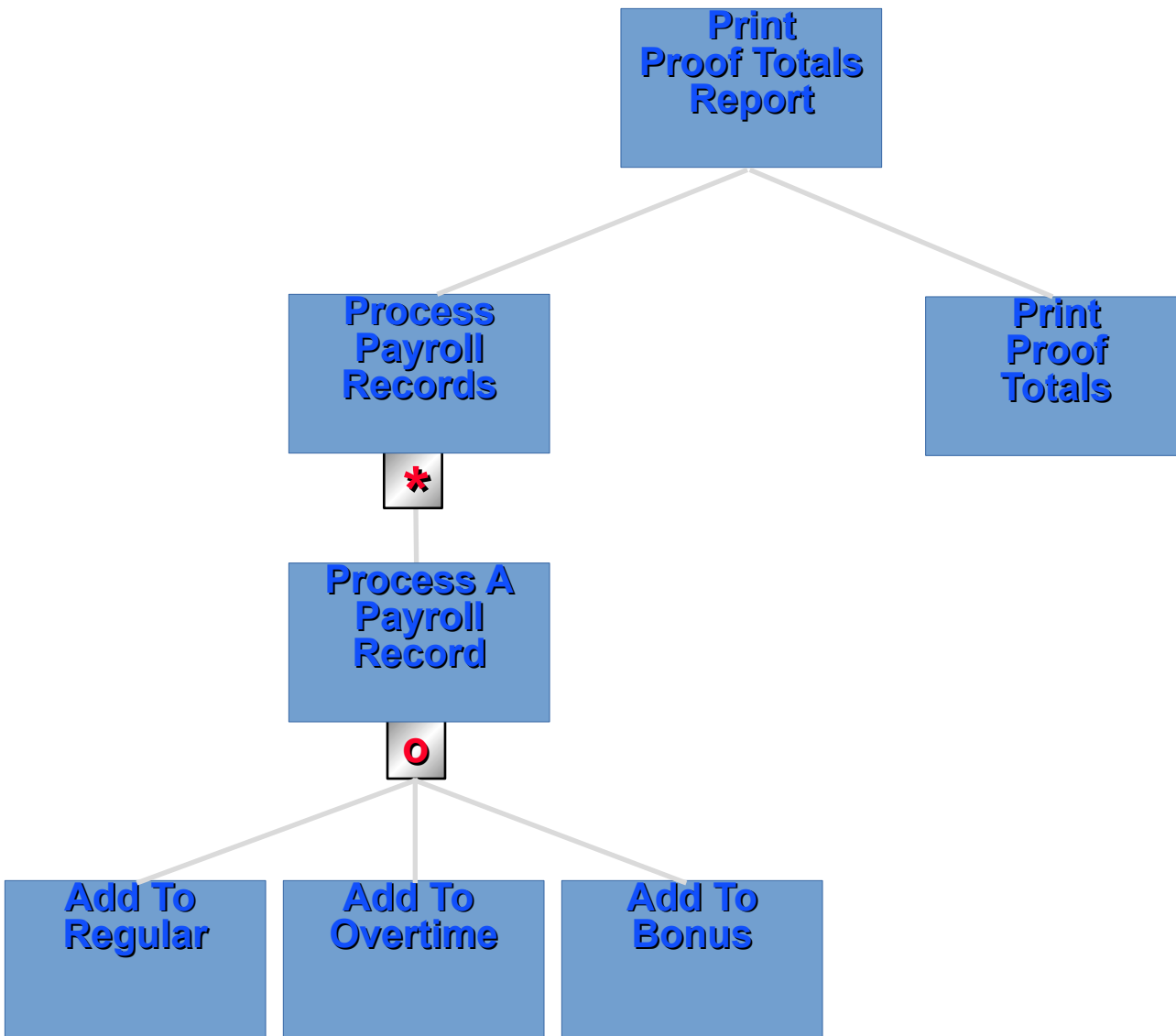
1. Define the problem completely
2. Work out what needs to be done to solve the problem.
3. Using Stepwise Refinement design a Program Structure Diagram (PSD) to represent your solution to the problem.

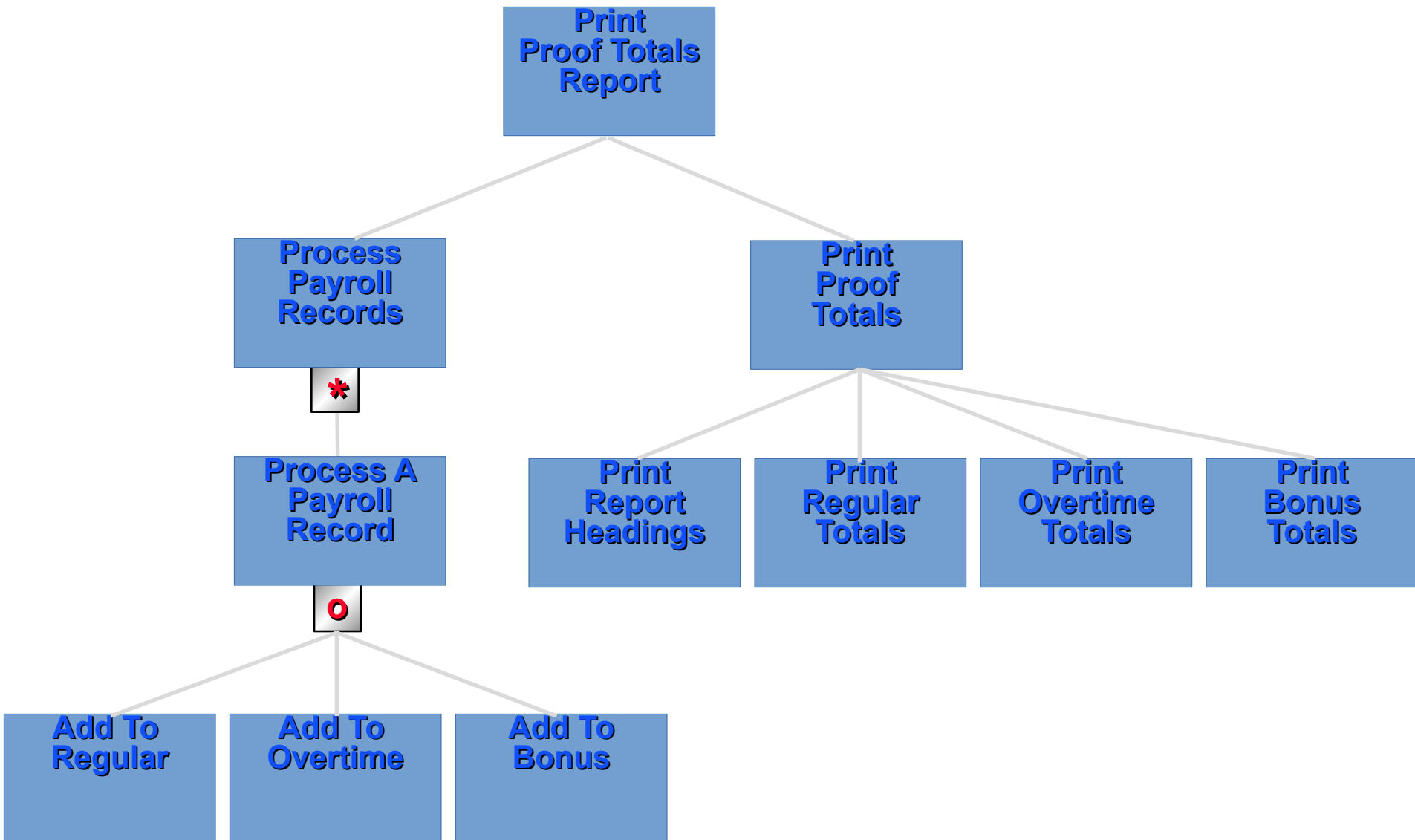


**Print
Proof Totals
Report**









4. Write out the Executable Operations needed to produce the output from the input.

1. OPEN OUTPUT Print-File.
2. CLOSE Print-File.
3. WRITE Print-Line FROM Heading-Line AFTER ADVANCING PAGE.
4. WRITE Print-Line FROM Regular-Line AFTER ADVANCING 1 LINE.
5. WRITE Print-Line FROM Overtime-Line AFTER ADVANCING 1 LINE.
6. WRITE Print-Line FROM Bonus-Line AFTER ADVANCING 1 LINE.
7. MOVE Regular-Total TO Print-Reg-Total.
8. MOVE Overtime-Total TO Print-Overtime-Total.
9. MOVE Bonus-Total TO Print-Bonus-Total.
10. MOVE Regular-Count TO Print-Reg-Count.
11. MOVE Overtime-Count TO Print-Over-Count.
12. MOVE Bonus-Count TO Print-Bonus-Count.
13. ADD Earnings TO Regular-Total.
14. ADD Earnings TO Overtime-Total.
15. ADD Earnings TO Bonus-Total.
16. ADD 1 TO Regular-Count.
17. ADD 1 TO Overtime-Count.
18. ADD 1 TO Bonus-Count.
19. READ Payroll-File
 AT END SET End-of-File TO TRUE
END-READ.
20. OPEN INPUT Payroll-File.
21. CLOSE Payroll-File.

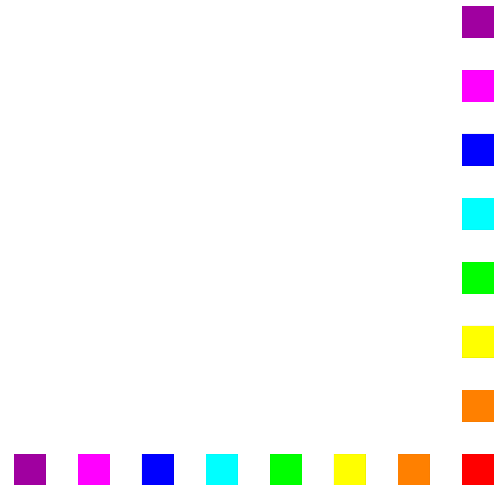
5. Assign the Executable Operations to the appropriate places in the PSD.

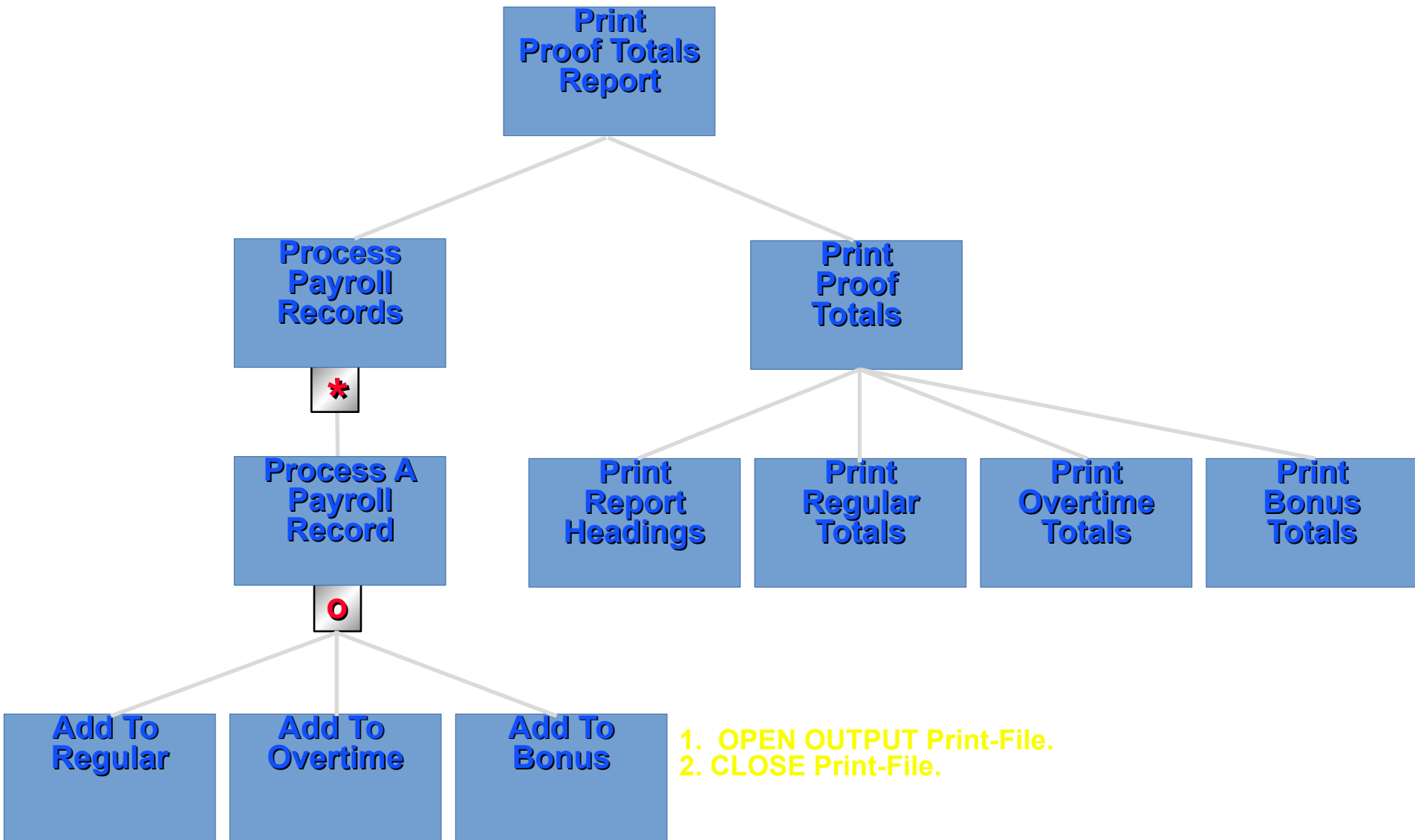
Algorithm

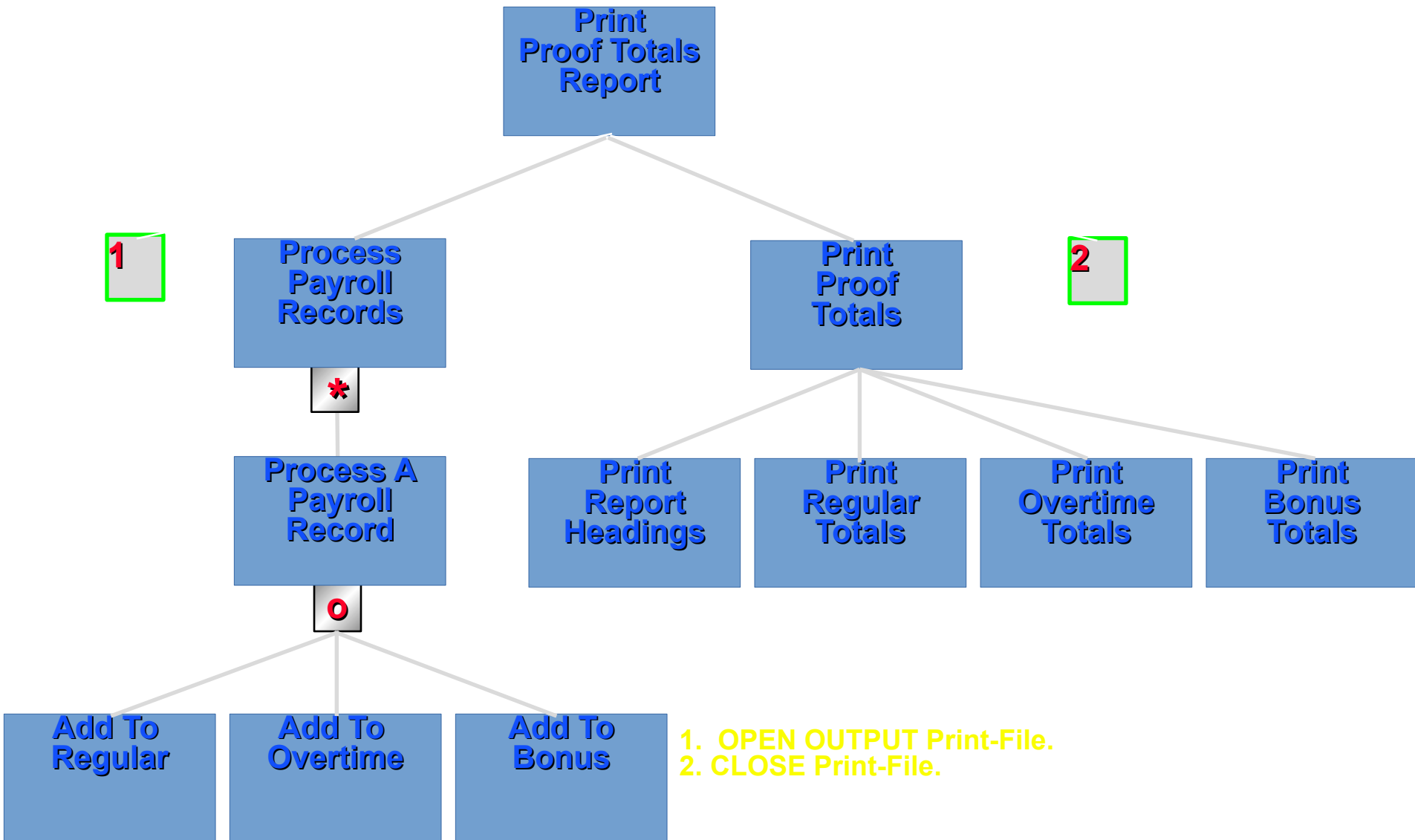
For all Executable Operations.

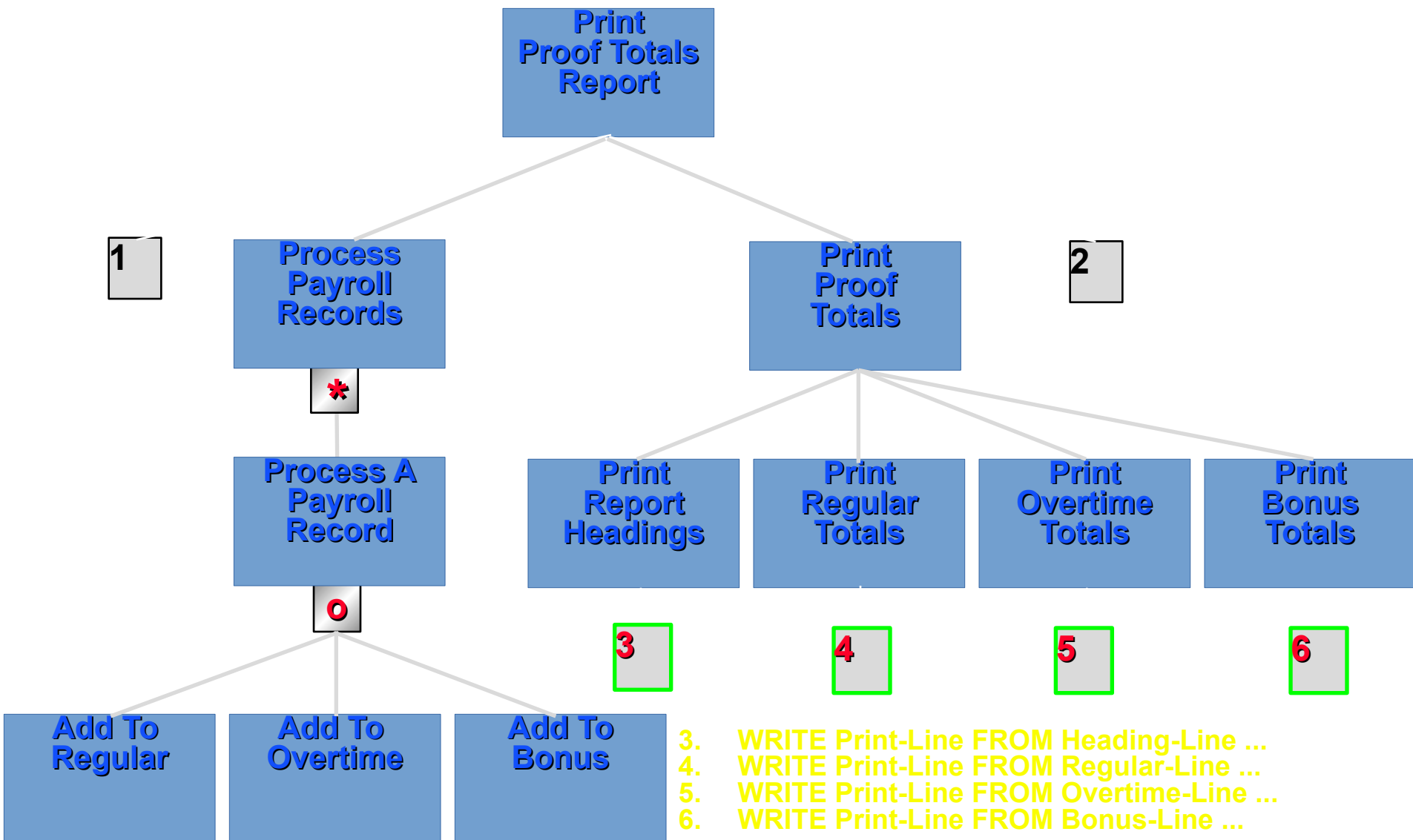
Get an Executable Operation.

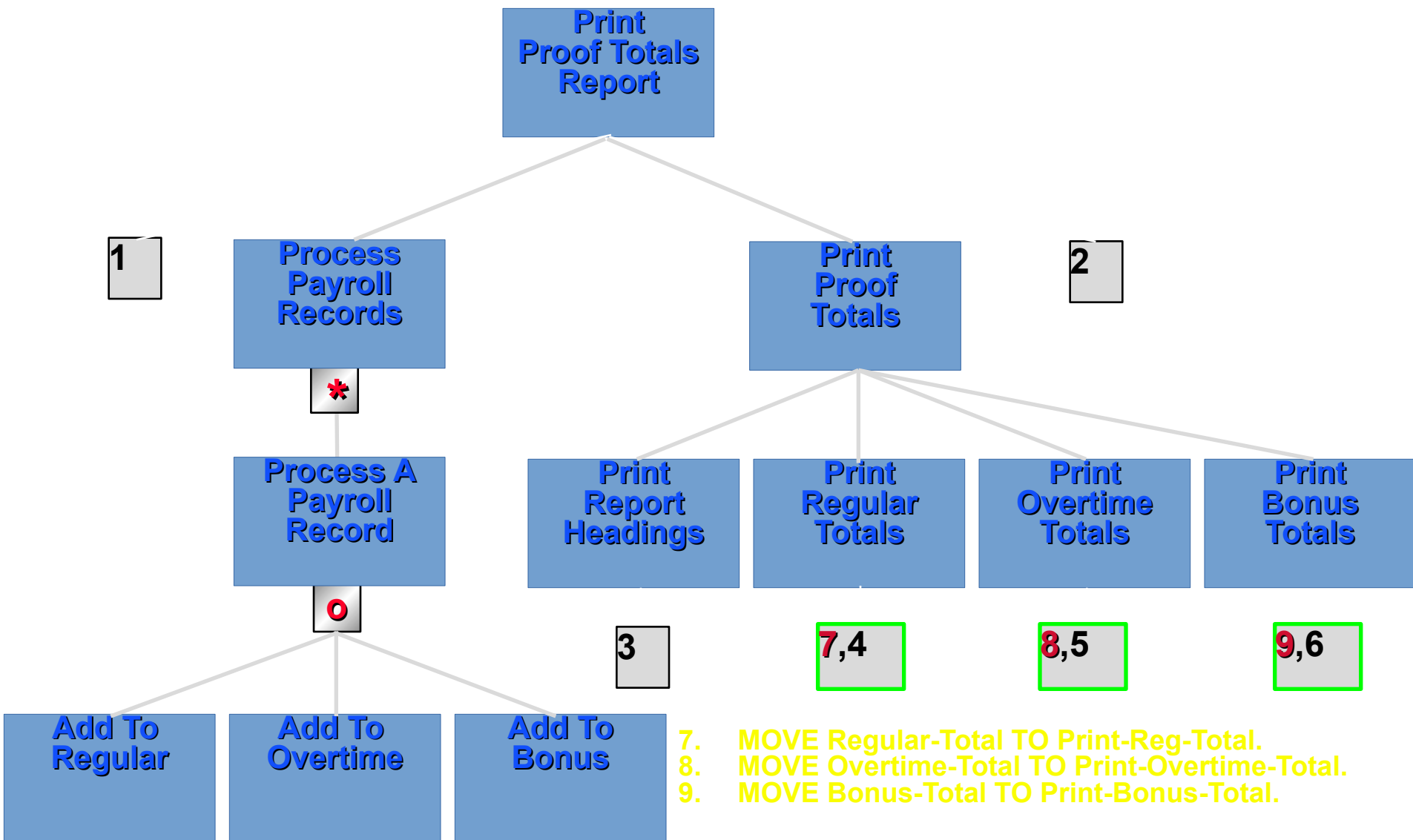
Ask "To what component
attach this operation
work?".

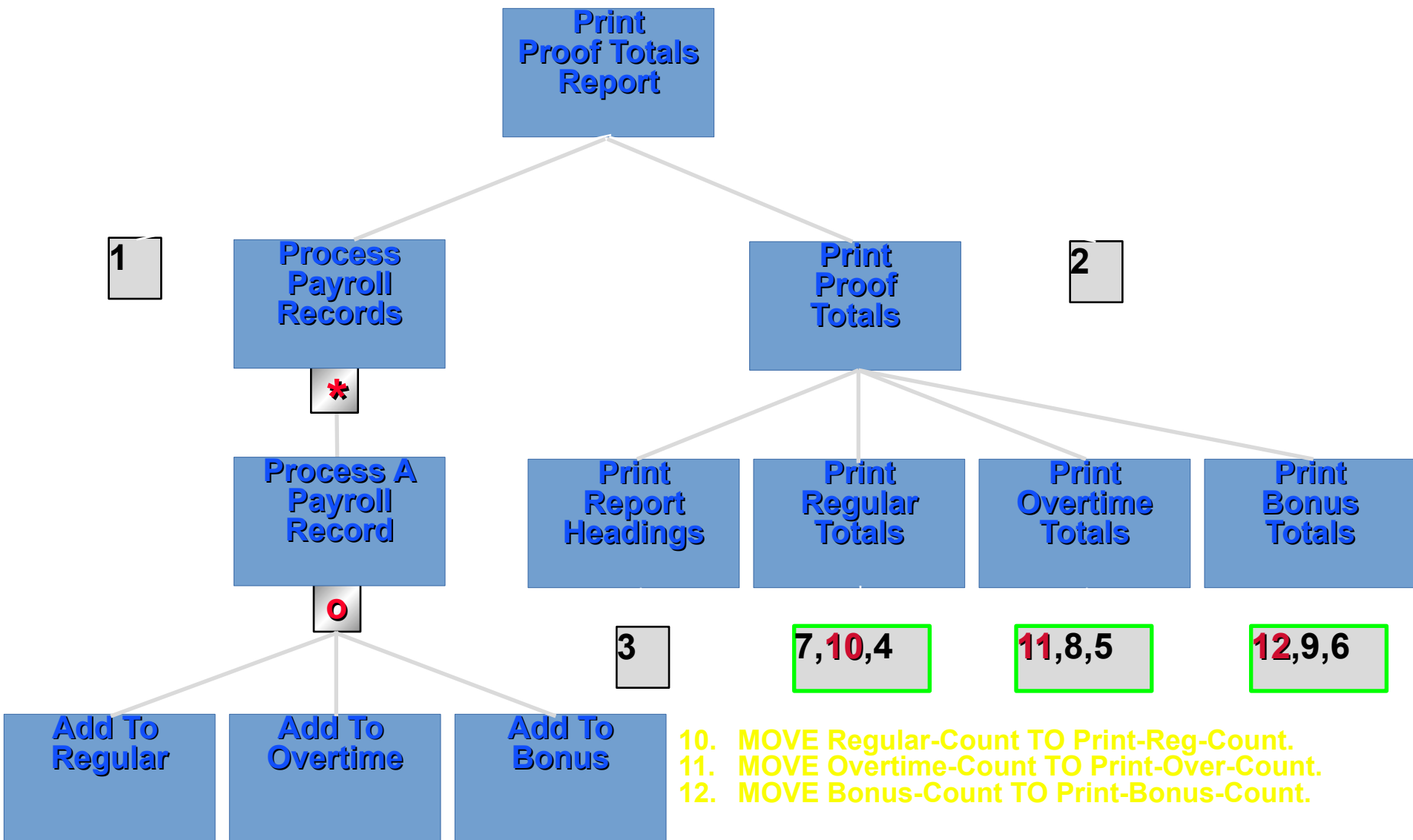


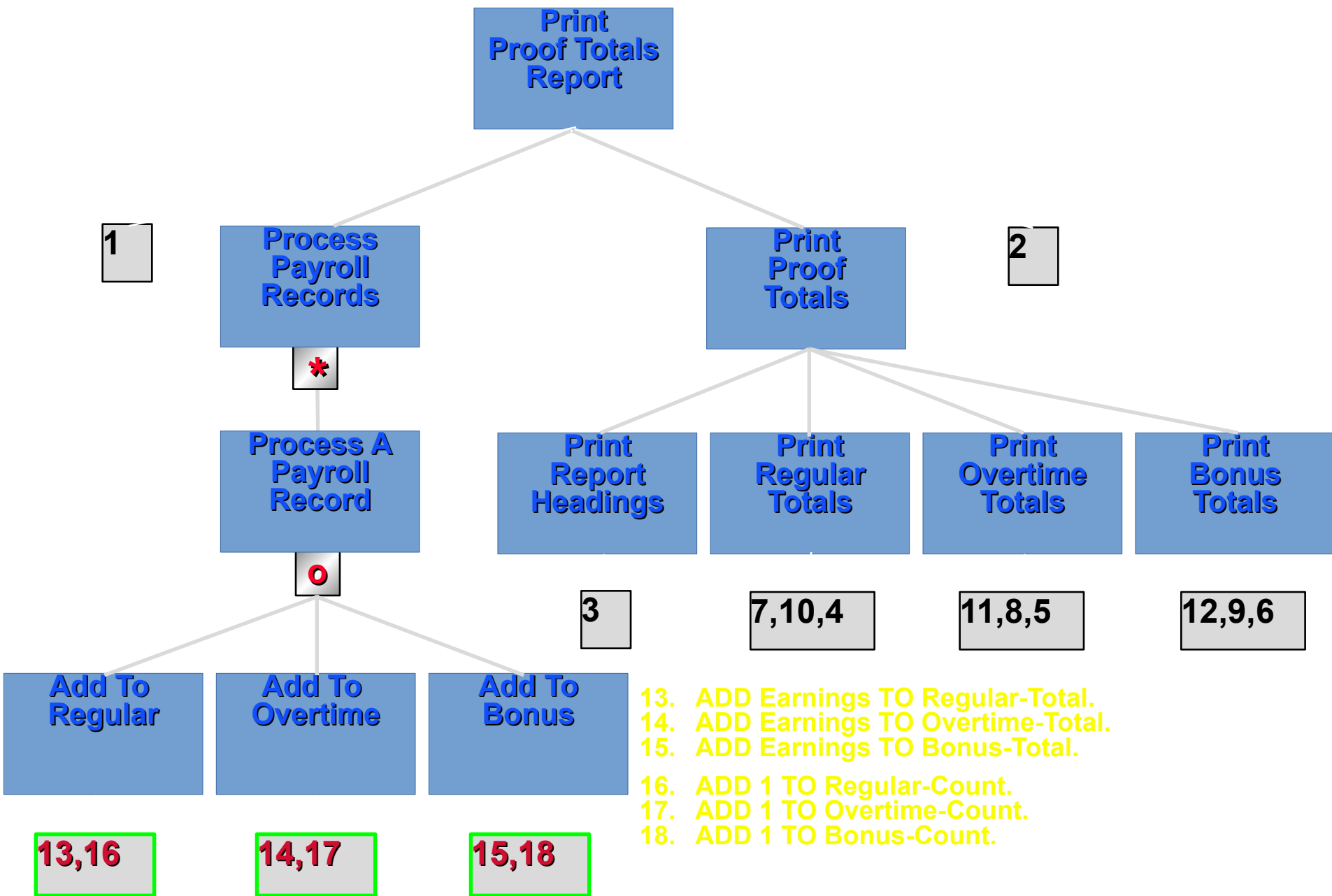


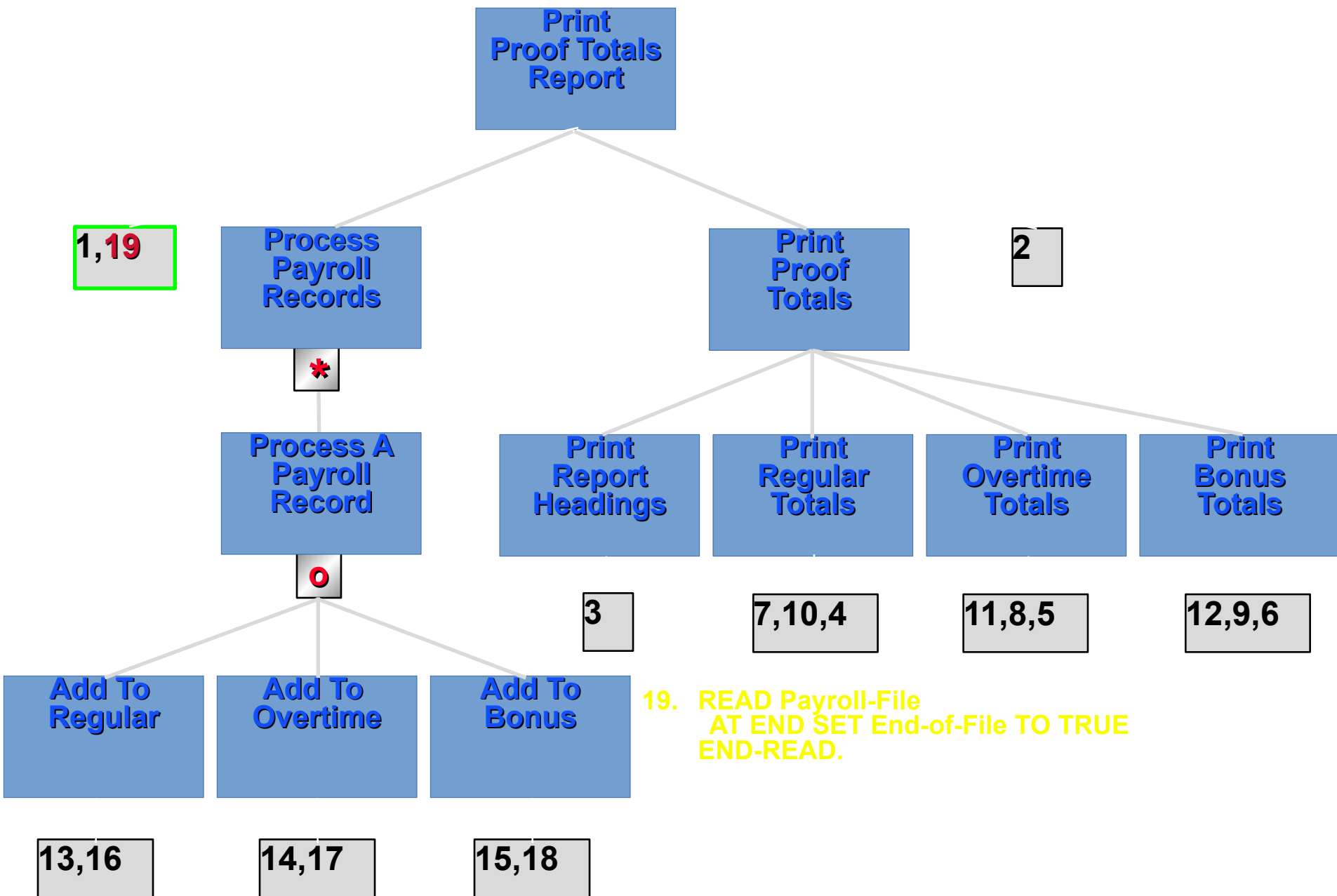


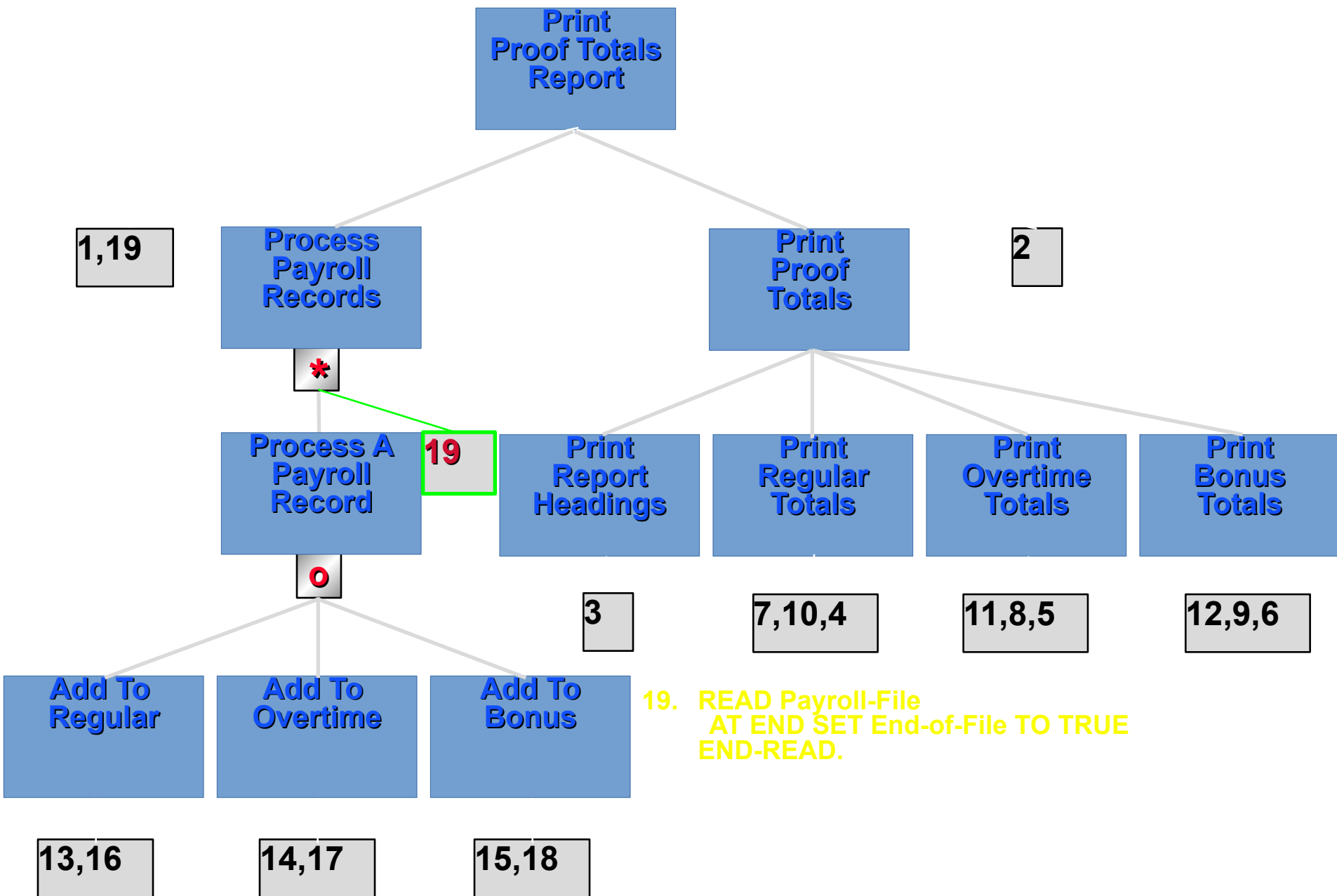


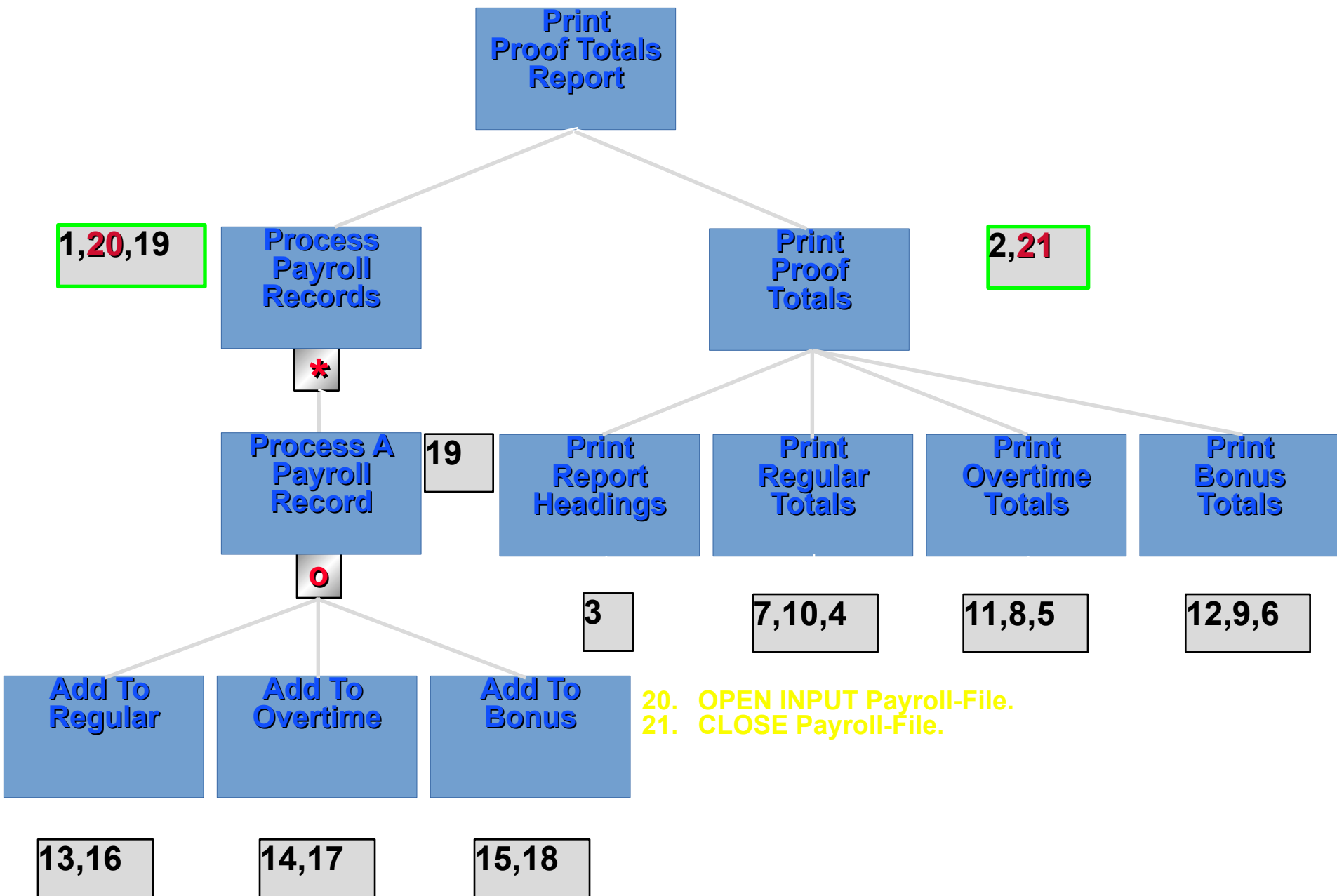












6. Insert the Iteration and Selection conditions.

Go to each iteration component and ask;

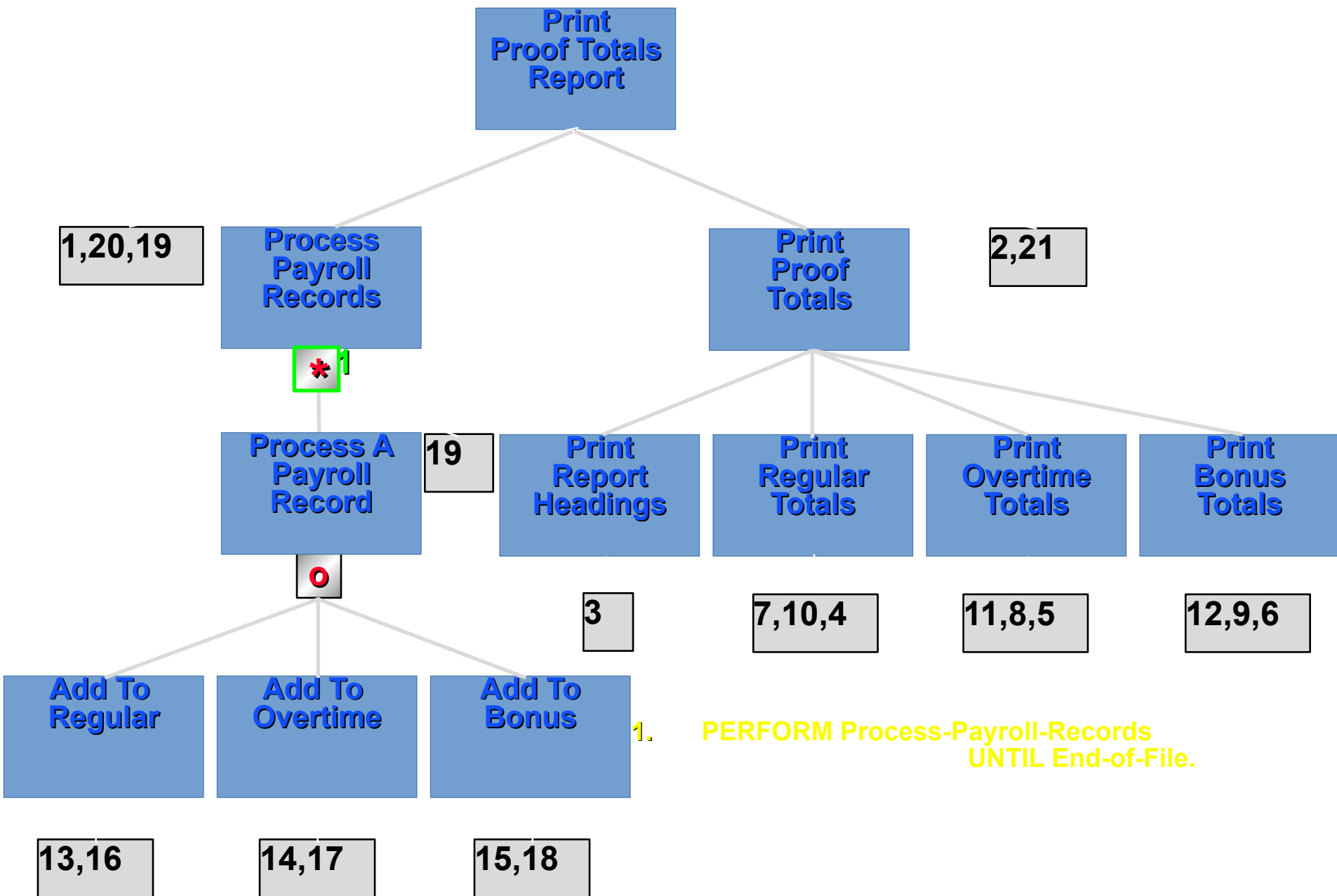
"What iteration statement must I use here to get the program to work correctly?".

Write the Iteration statement, give it a number and insert its number into the diagram.

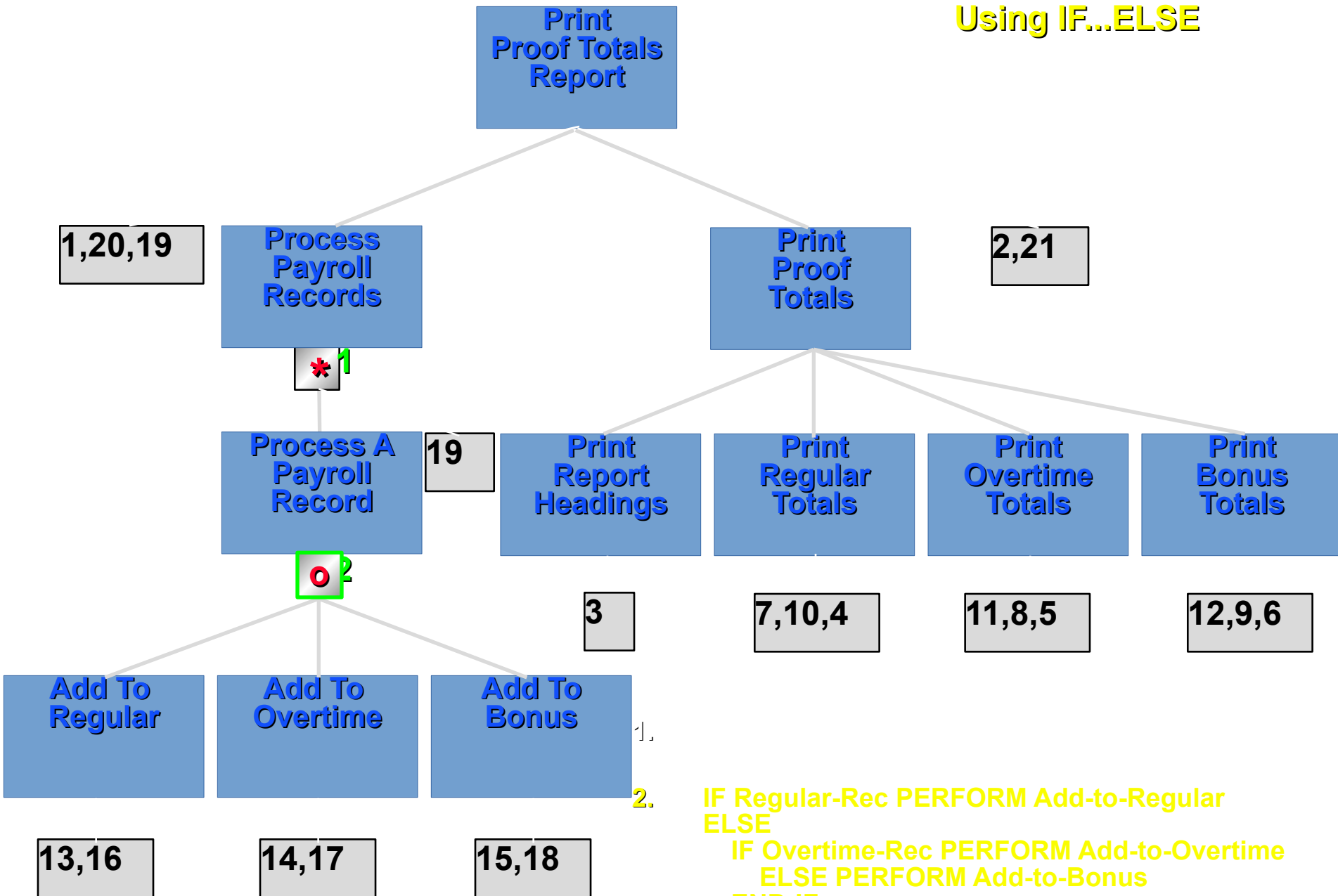
Go to each selection component and ask;

"What selection statement must I use here to get the program to work correctly?".

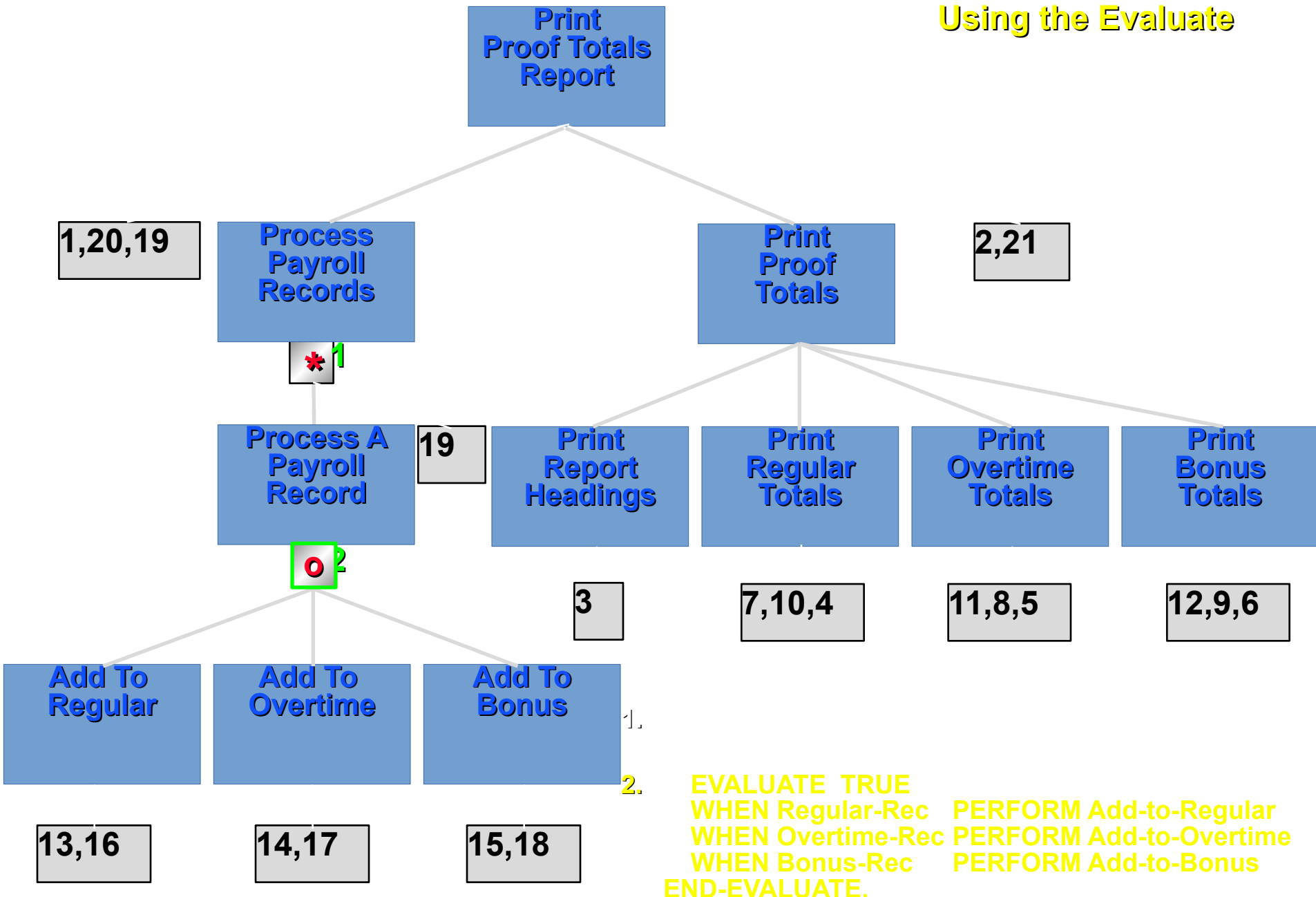
Write the Selection statement, give it a number and insert its number into the diagram.



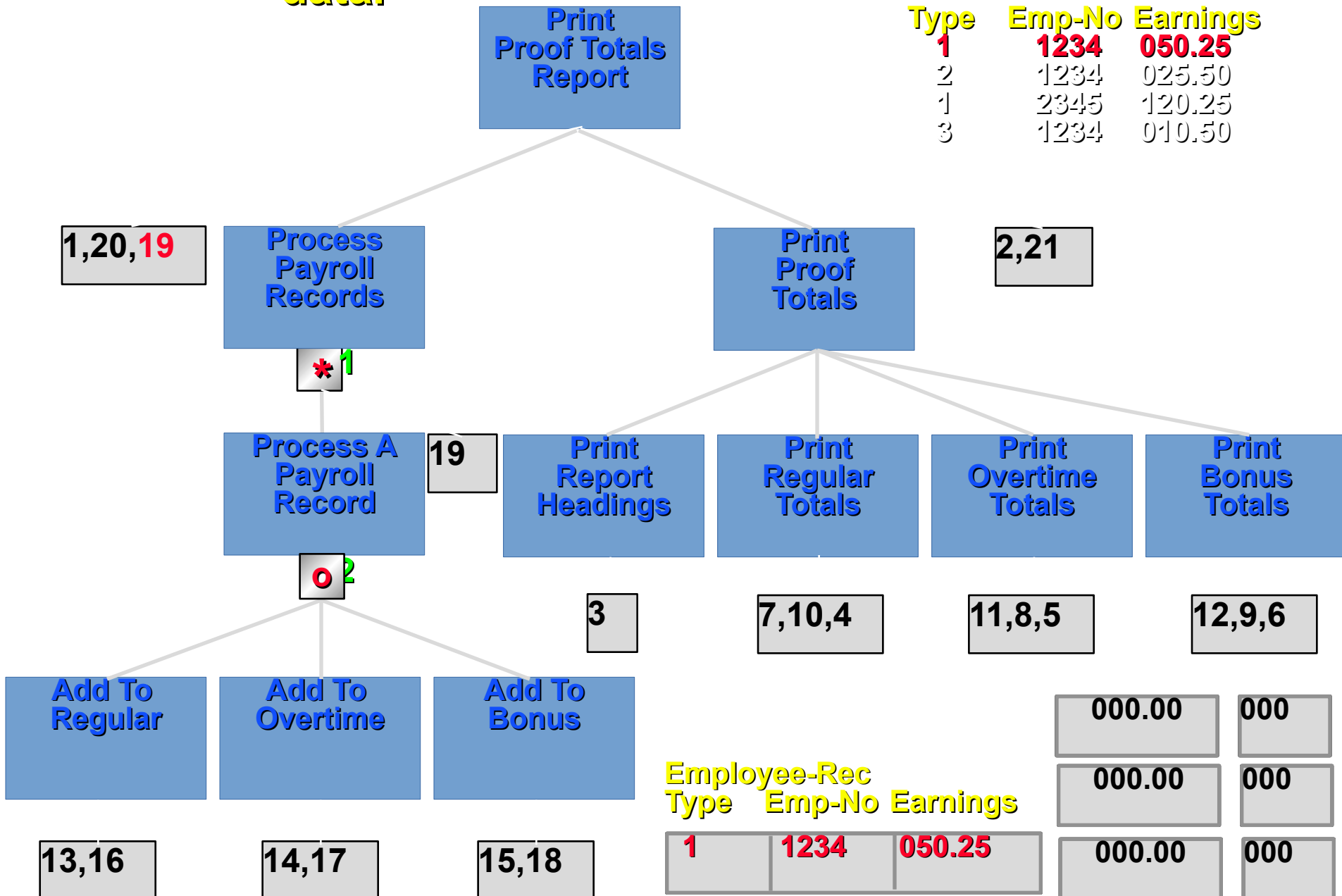
Using IF...ELSE



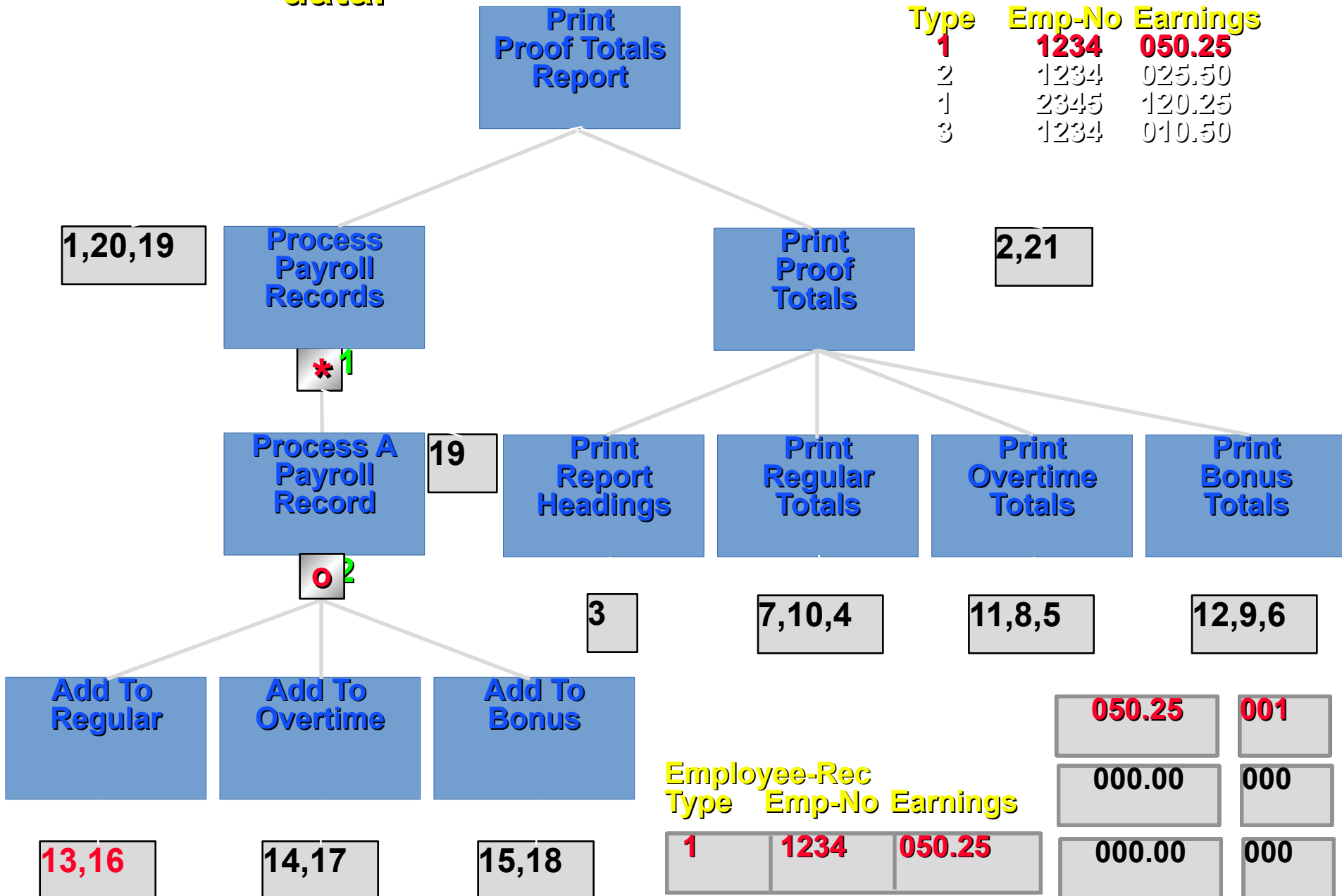
Using the Evaluate



7. Test the program using test data.



7. Test the program using test data.



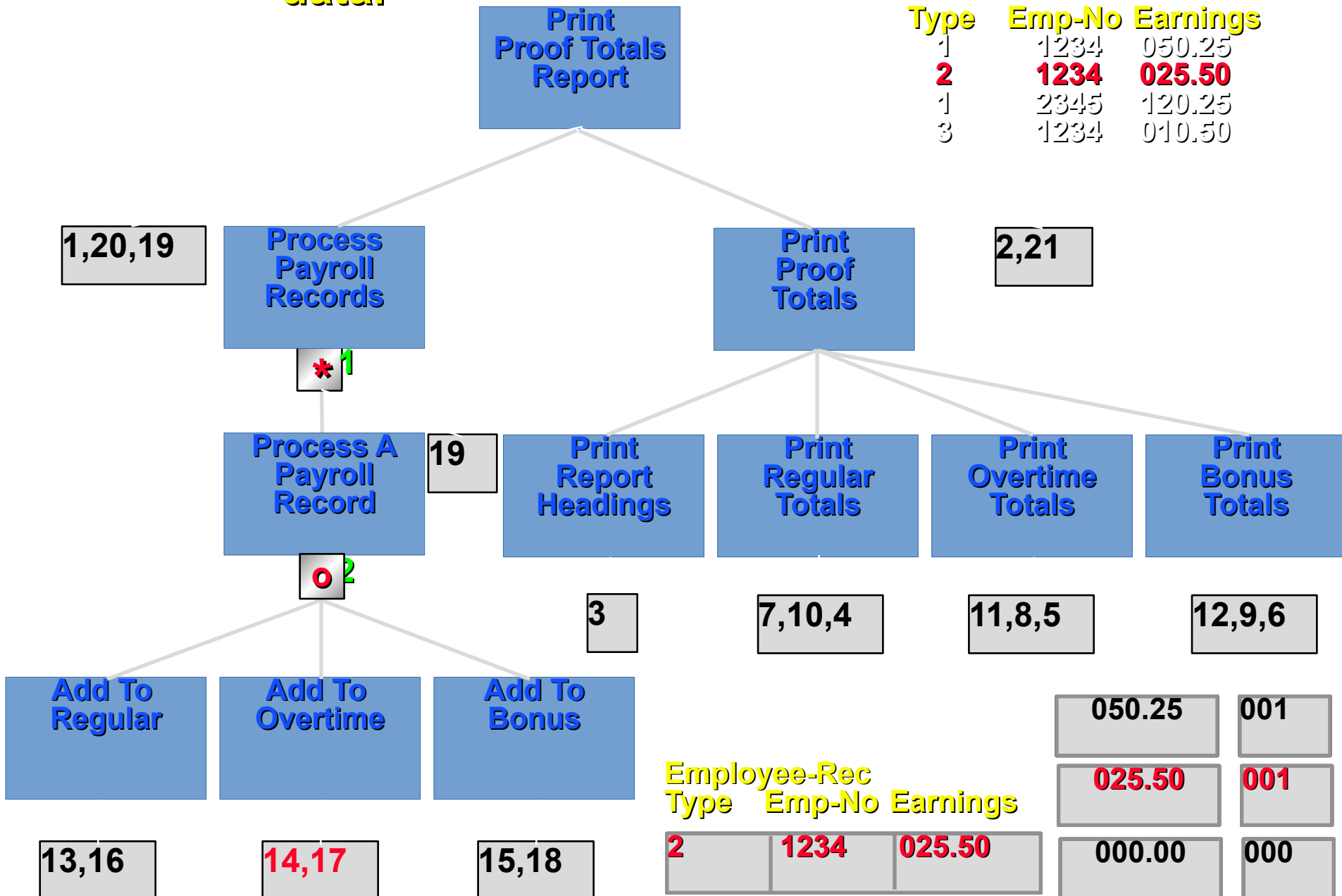
data.



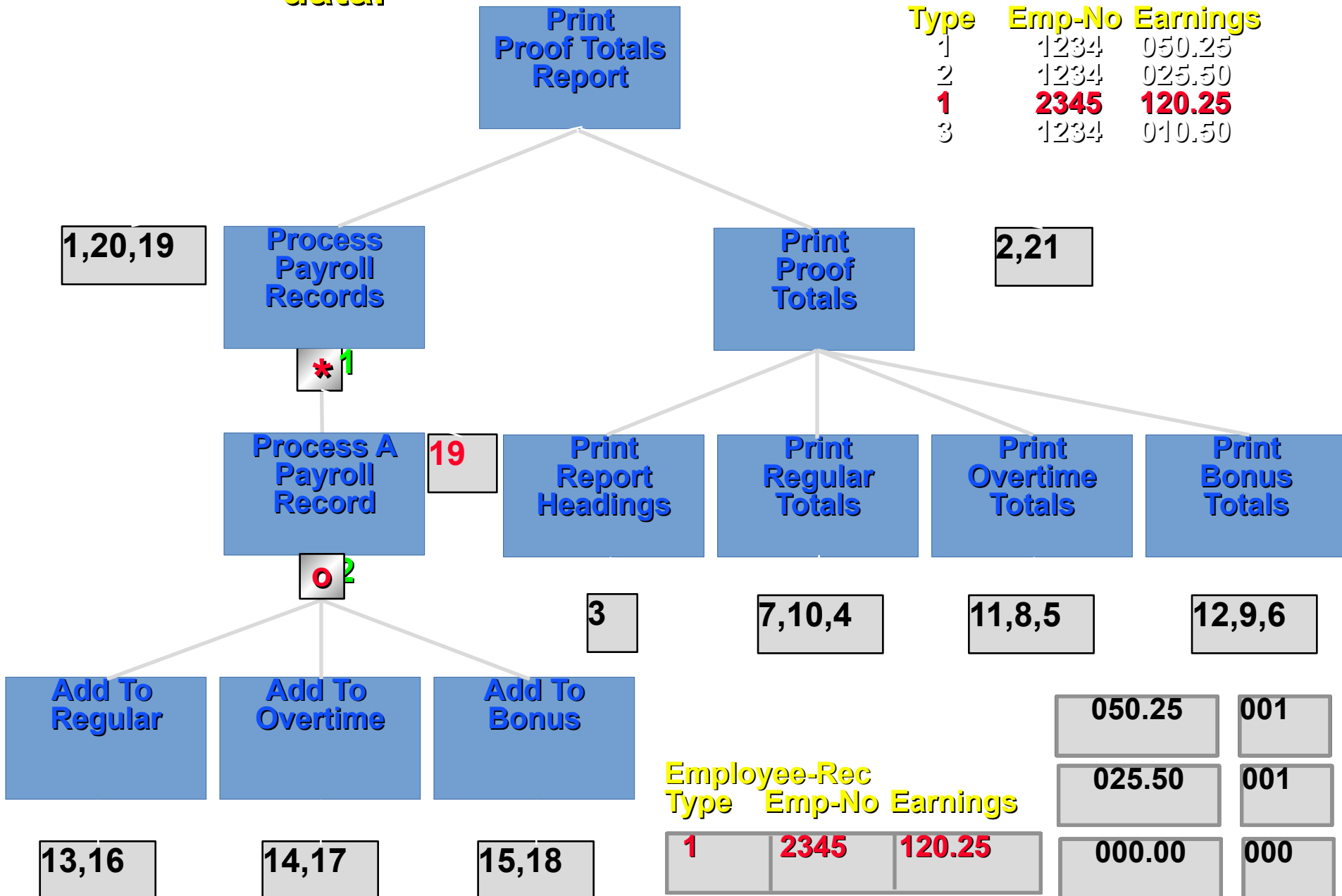
2	1234	025.50
---	------	--------

050.25	001
000.00	000
000.00	000

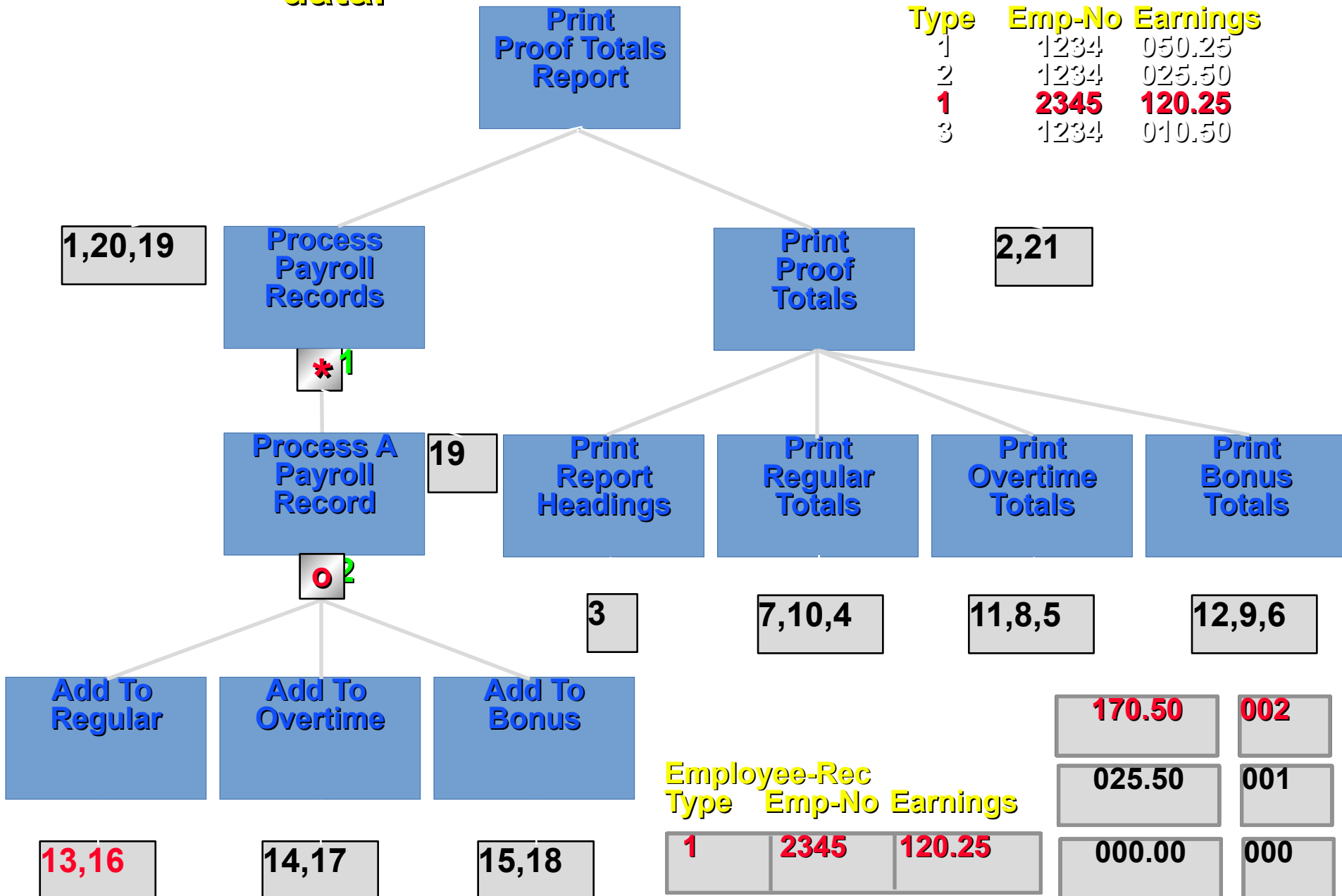
7. Test the program using test data.



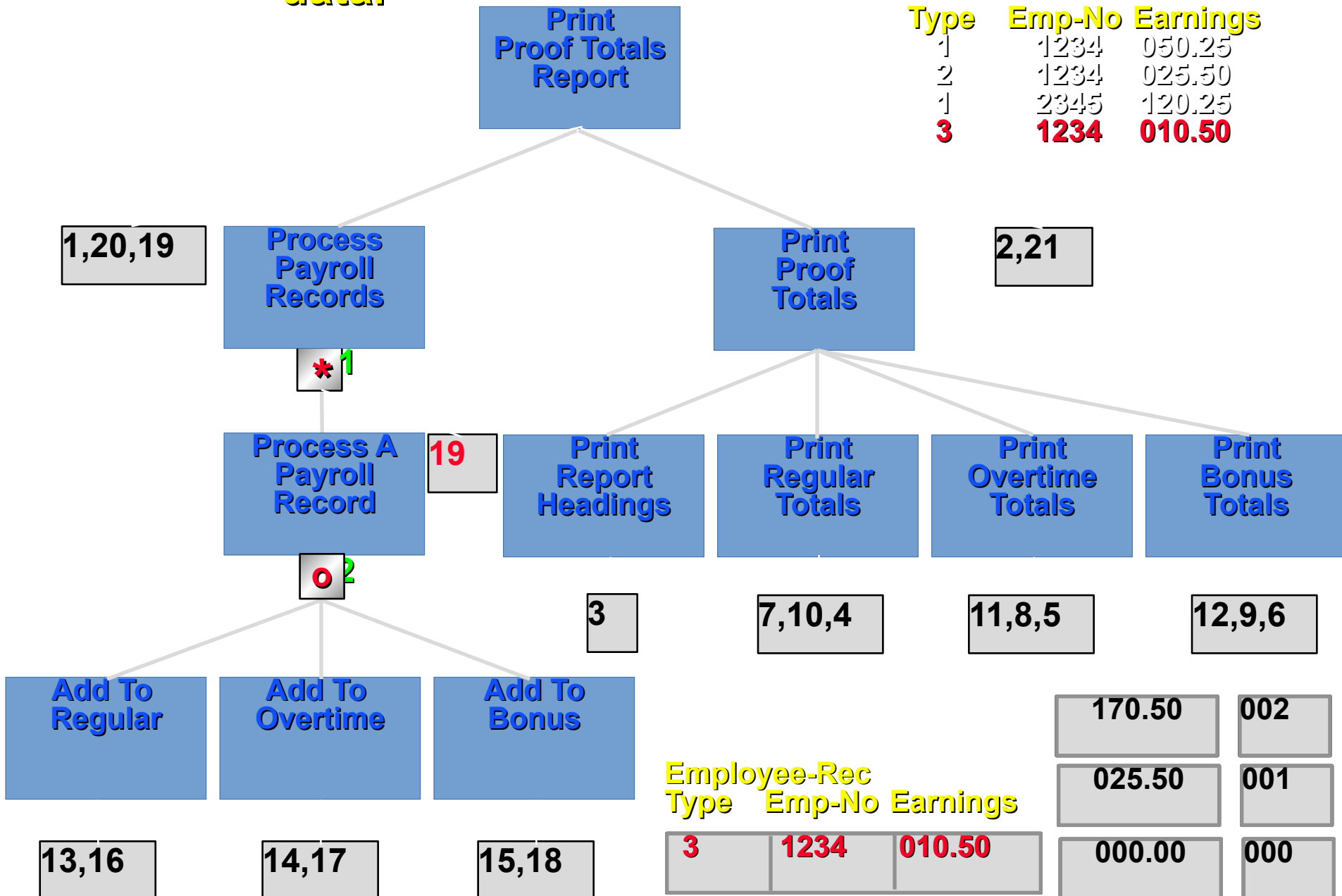
7. Test the program using test data.



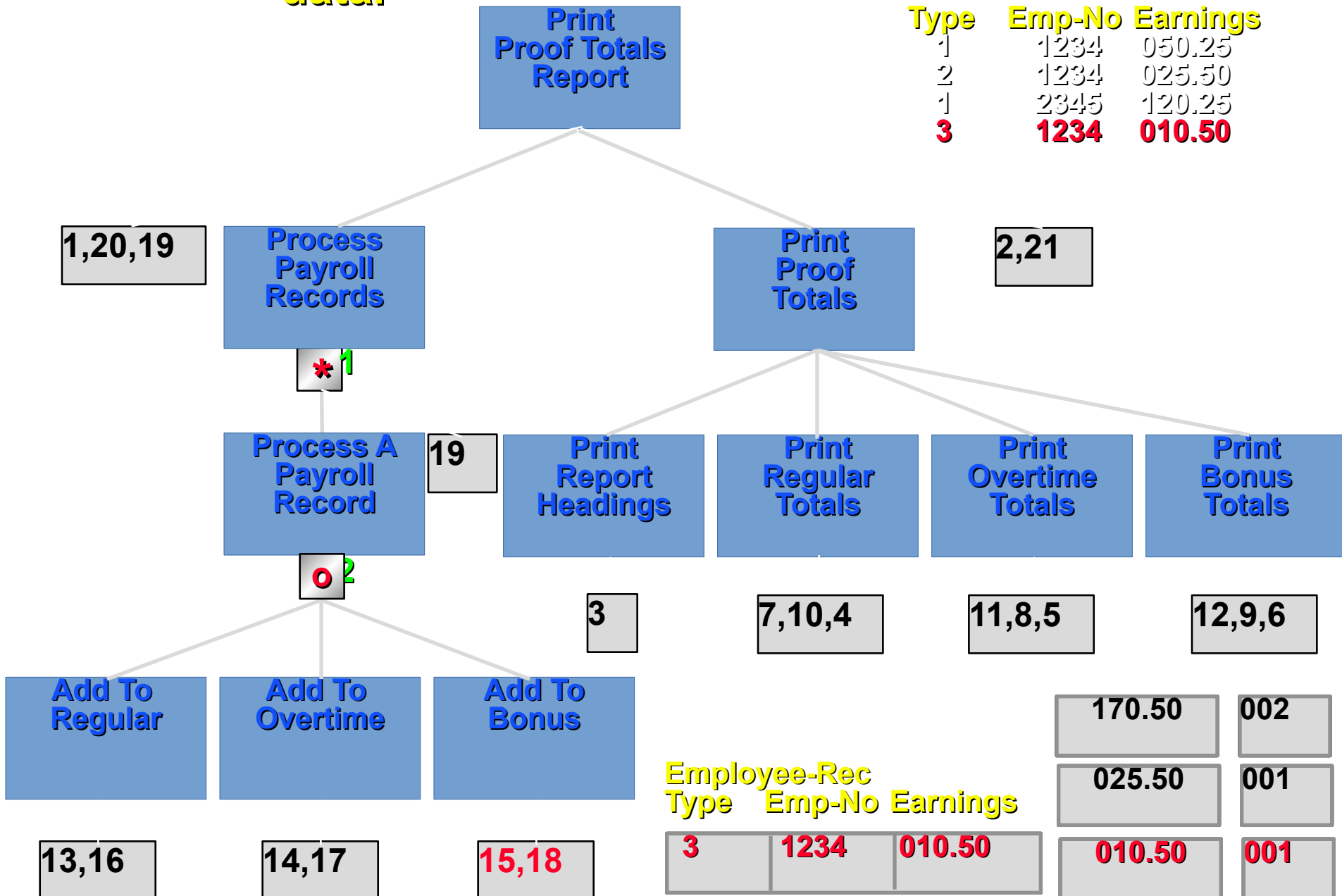
7. Test the program using test data.



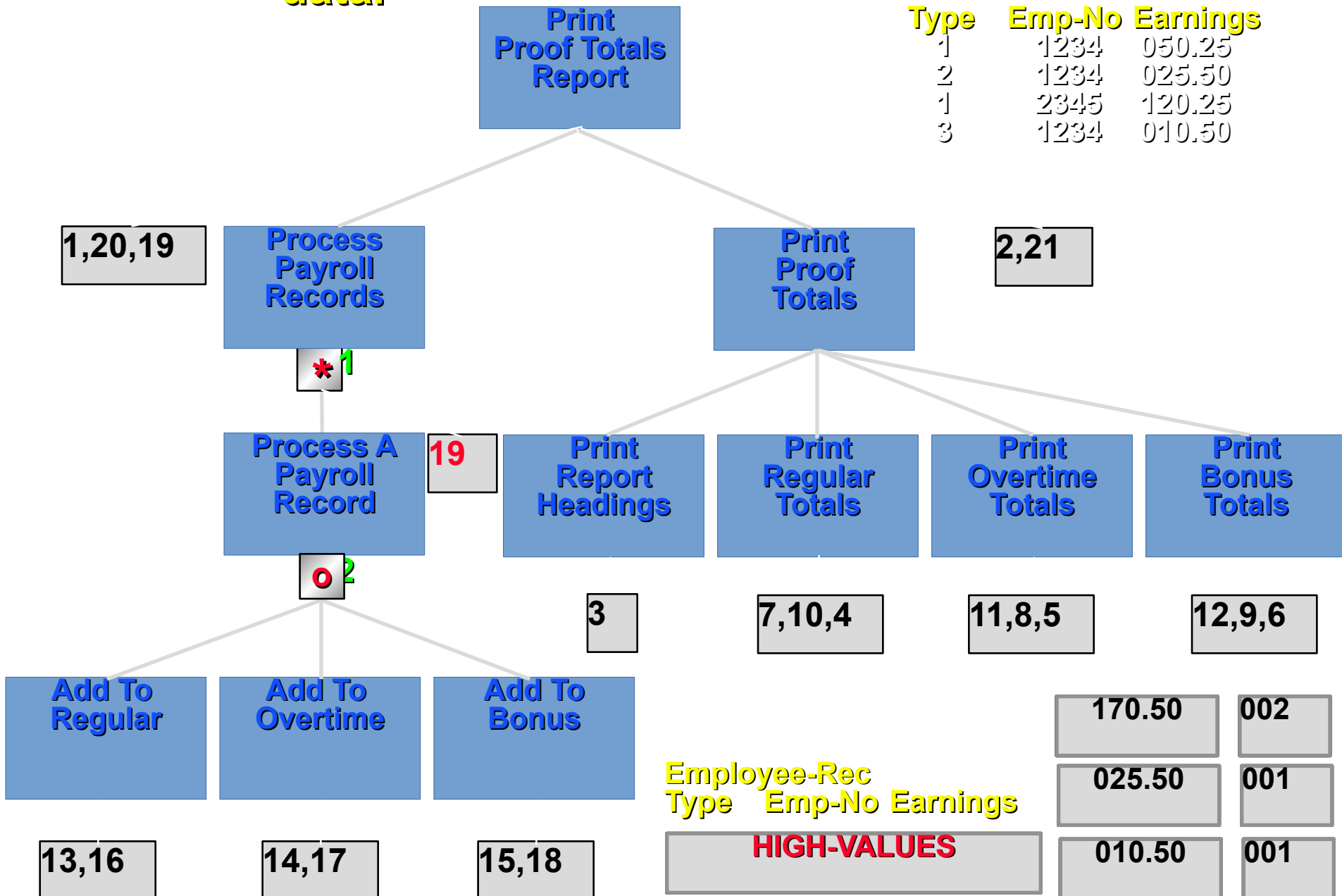
7. Test the program using test data.



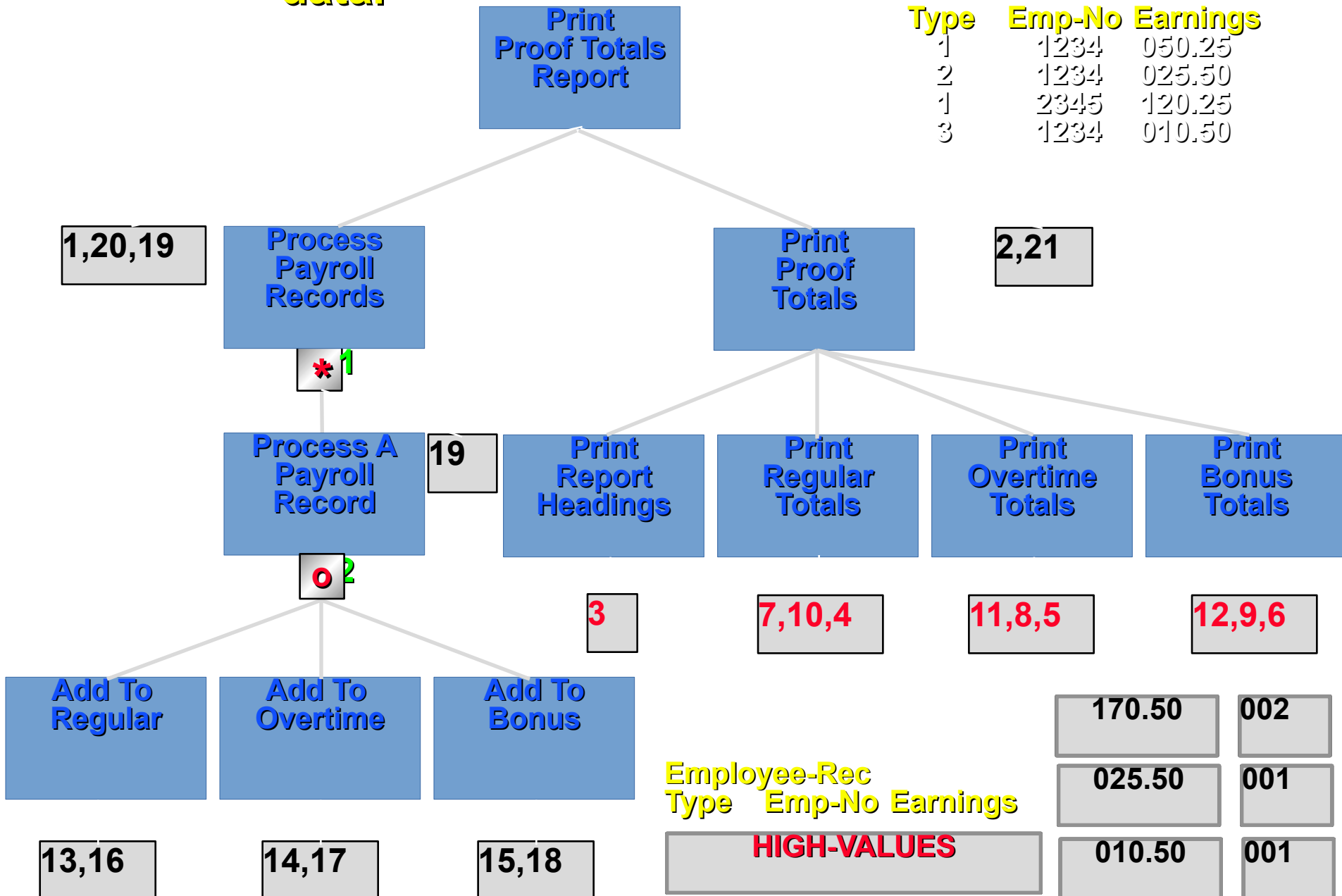
7. Test the program using test data.



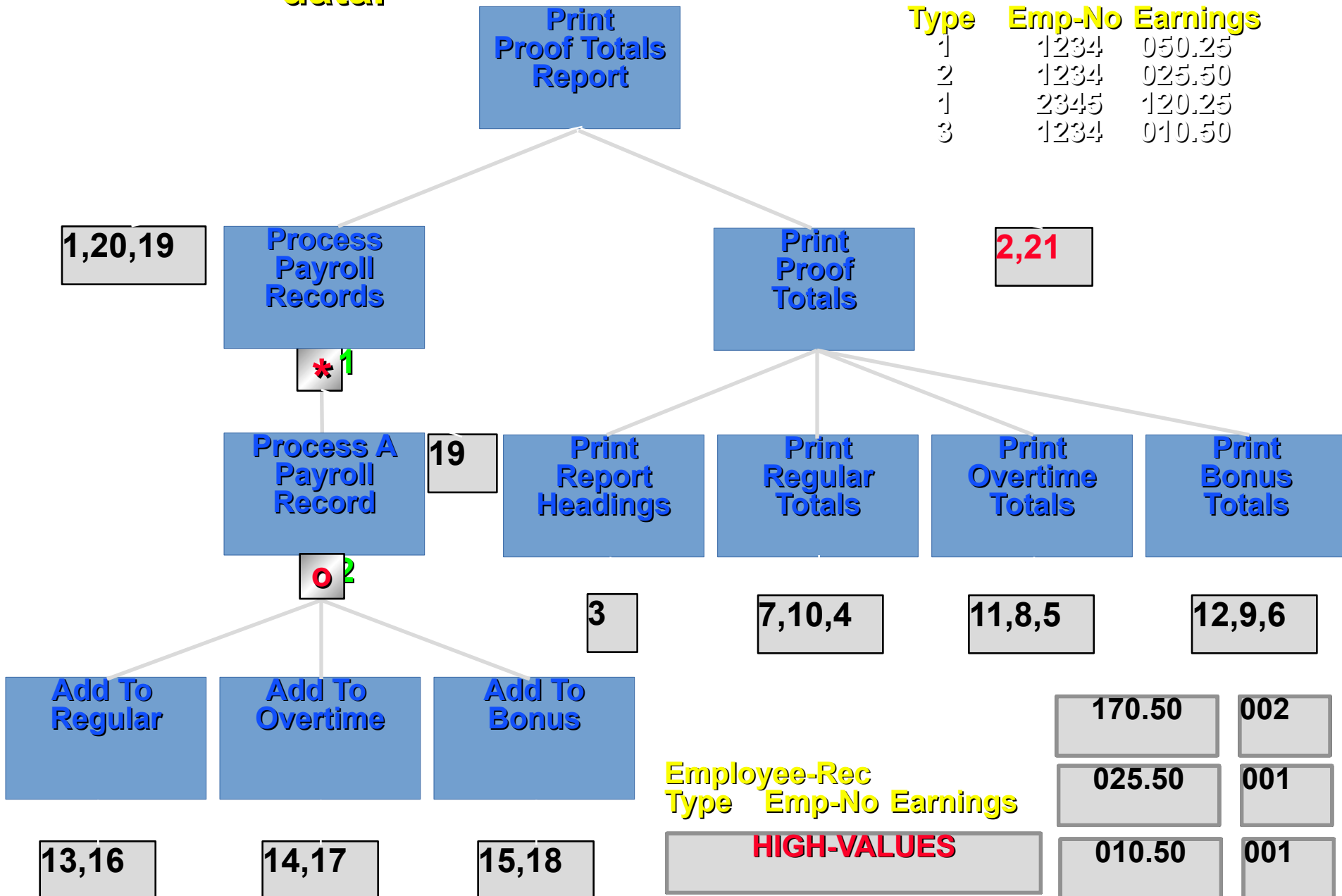
7. Test the program using test data.



7. Test the program using test data.



7. Test the program using test data.



8. Translate the populated PSD into COBOL code

Start at the top of the diagram.

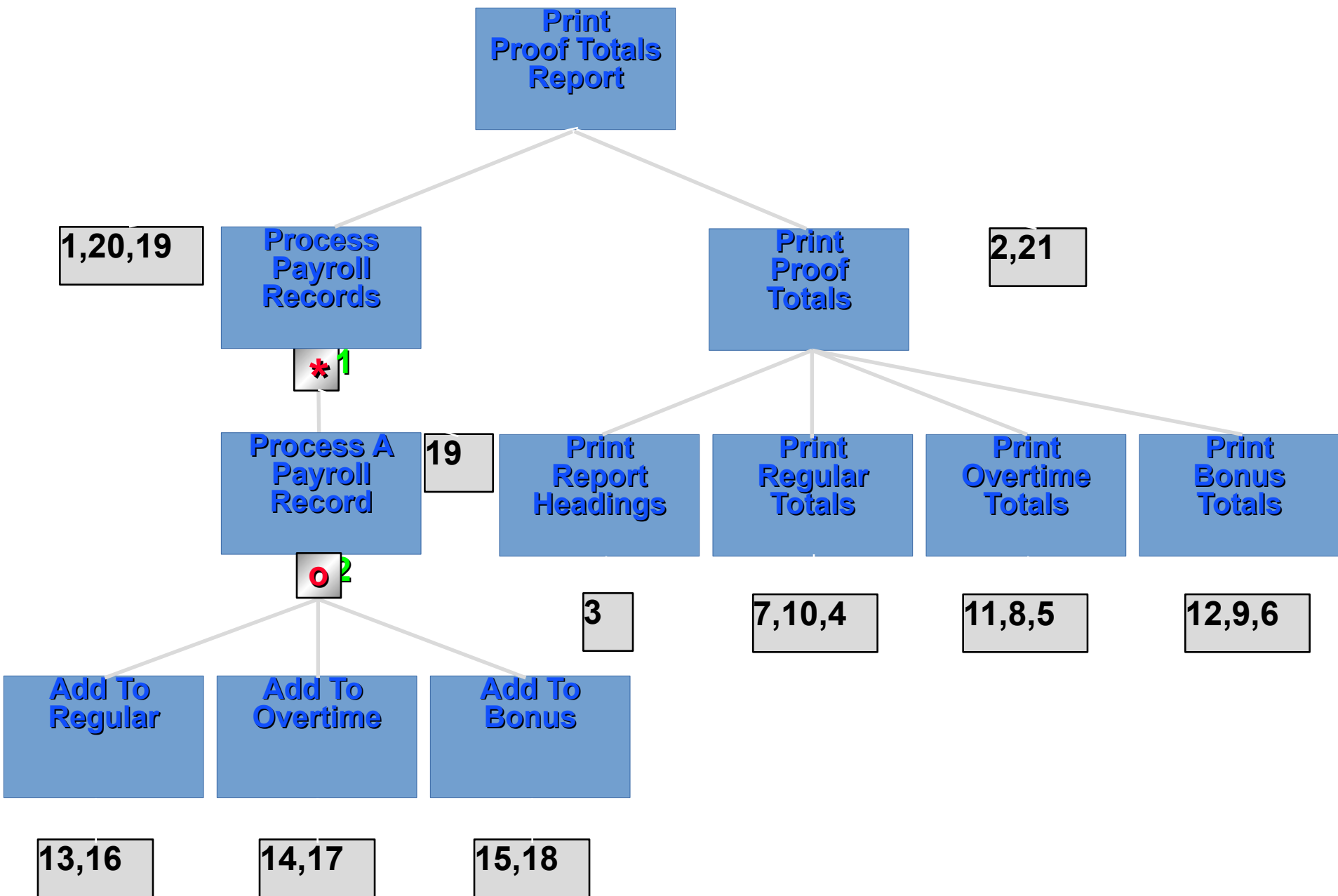
At each parent component write the sequence of executable operations, child components and iteration and selection conditions as statements in a paragraph.

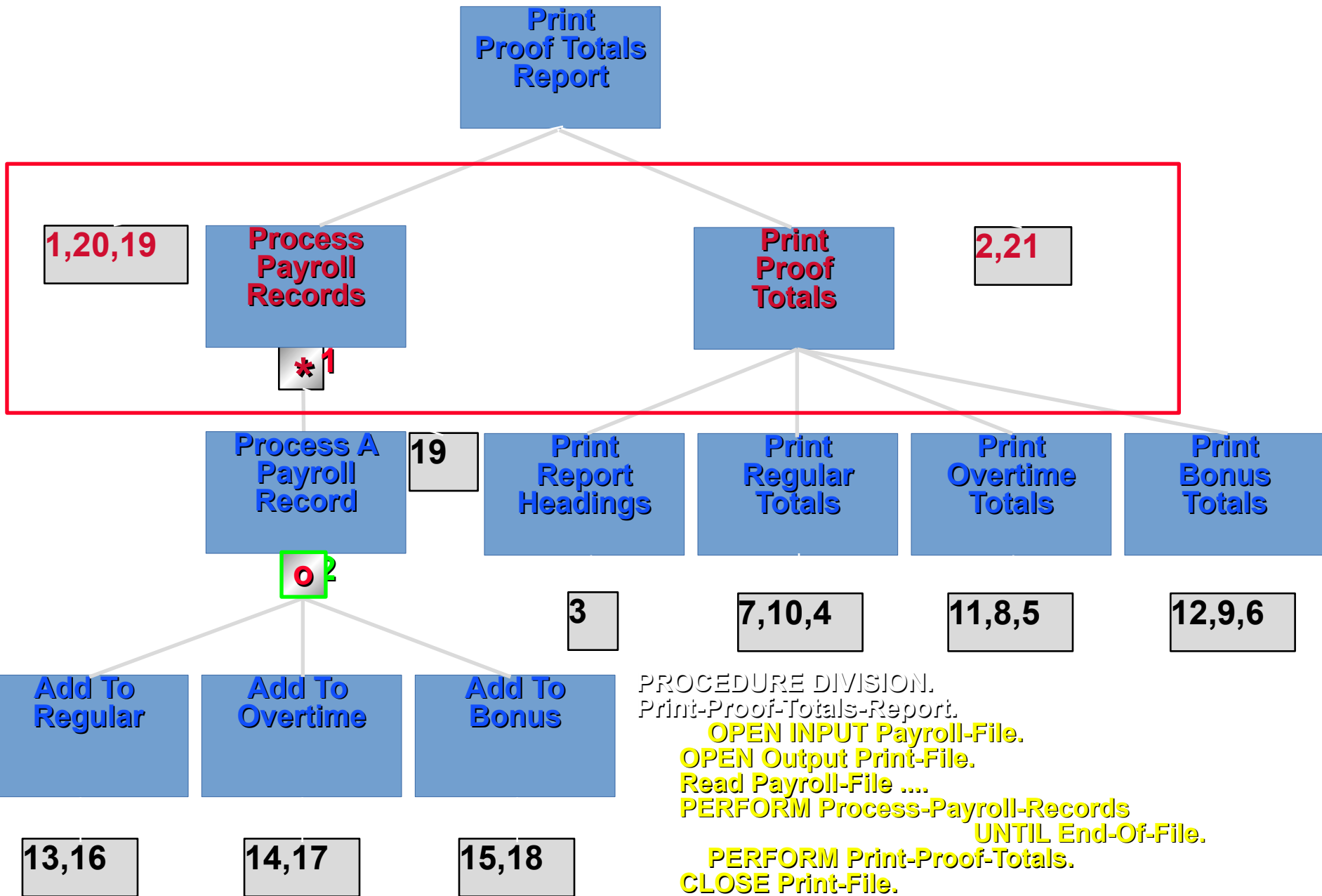
Replace child components with PERFORMs of the component name.

At the top component, use the name as the name of the first paragraph and precede it with the PROCEDURE DIVISION header.

End the sequence with the STOP RUN statement.







PROCEDURE DIVISION.
 Print-Proof-Totals-Report.
 OPEN INPUT Payroll-File.
 OPEN Output Print-File.
 Read Payroll-File
 PERFORM Process-Payroll-Records
 UNTIL End-Of-File.
 PERFORM Print-Proof-Totals.
 CLOSE Print-File.
 CLOSE Payroll-File.
 STOP RUN.

